BESPOKE

From customer words and moodboards
to garment specifications a tailor can cut from

Research and engineering plan

Text Analytics and Big Data Analytics

[Student Name 1] [Student Name 2] [Student Name 3] [Student Name 4]

Guide: [Guide Name]

Amrita School of Artificial Intelligence, Faridabad

October 2026

Summary

Custom clothing in India is specified in chat messages and phone photos, and the tailor's memory of that conversation is what reaches the cutting table. About 12 million people work as custom tailors in India, yet no software turns a customer's description and inspiration images into instructions a tailor can execute.

BESPOKE does that. A customer writes in English or Hinglish and builds a moodboard on an embedded Excalidraw canvas. A language engine turns the words into part-level, calibrated values; a moodboard engine reads the whole board as a graph of images, circled regions and notes, including rejected references and comparisons such as "deeper than #1 but less than #2". A resolver merges the evidence into a versioned DesignSpec, asks only the questions whose wrong answer would be expensive, and renders a tailor sheet in English and Hindi. A Spark platform over more than 100 million public retail records and the order log learns trends, defaults and suggestions.

A review of fifteen papers from IEEE, Springer, Elsevier and Taylor & Francis (2022–2026) finds that the closest systems (ChatGarment, Design2GarmentCode, AIpparel, GenPattern, MuJo-SF) accept one clean input, flatten or silently complete the garment, ignore code-mixed language and Indian garments, never ask, and are never judged by tailors. The plan sets three objectives against that gap, four research questions, a benchmark (BESPOKE-SPEC), measurable targets, and a nine-week timeline that starts with a working slice built in the first two hours.

Contents

1 Introduction 1

1.1 The problem 1

1.2 What an earlier prototype taught us 2

1.3 From retrieval to specification 2

1.4 Contributions 3

1.5 How to read this plan 3

2 Literature Review 4

2.1 Search protocol 4

2.2 Attribute extraction from product text and images 4

2.3 Customer language and fashion text 5

2.4 Fashion vision and Indian garments 6

2.5 From design intent to garment patterns 6

2.6 Big data for fashion 7

2.7 Work beyond the five publishers 7

2.8 What the literature leaves open 8

3 Research Gap and Objectives 10

3.1 The five closest systems 10

3.2 The gap 11

3.3 Objectives 11

3.4 Research questions 11

3.5 Scope 12

4 Models and Datasets 13

4.1 Language models 13

4.2 Vision models 13

4.3 Datasets 14

5 System Design 16

5.1 Architecture 16
5.2 The ontology 16
5.3 The DesignSpec 18
5.4 The resolver 18

6 The Language Engine 20
6.1 The pipeline 20
6.2 Normalisation: morphology and variants 20
6.3 Collocations, bag of words and n-grams 20
6.4 HMM and CRF taggers 21
6.5 How BERT tags a request 22
6.6 Binding and grounding 24
6.7 Small LLM with schema-constrained decoding 25
6.8 Conversation state 25
6.9 Calibration 25
6.10 Other syllabus tasks in the engine 25

7 The Moodboard Engine 27
7.1 Why Excalidraw 27
7.2 Reading the board as a graph 27
7.3 Evidence from each element 28
7.4 Pooling 28
7.5 Comparisons between references 29
7.6 The whole-board baseline 29
7.7 Importing inspiration 30

8 Studio and Tailor Sheet 32
8.1 The studio 32
8.2 The tailor sheet 32
8.3 Sewing patterns, later 32

9 The Big Data Platform 34
9.1 Why this needs big data 34
9.2 Layout of the lake 34
9.3 Scala: the domain in functional style 35
9.4 MapReduce: attribute co-occurrence 36

9.5 DataFrames, lazy evaluation and the DAG 36
9.6 MLlib pipelines 37
9.7 From data to suggestions 38
9.8 Privacy 39

10 Engineering Workflow 40
10.1 Repository 40
10.2 Services 40
10.3 API reference 41
10.4 Events 42
10.5 Quality gates 42

11 Evaluation Plan 43
11.1 The benchmark: BESPOKE-Spec 43
11.2 Metrics 43
11.3 Baselines and ablations 44
11.4 The tailor study 44
11.5 Targets 45
11.6 Threats to validity 45

12 Phases and Timeline 46
12.1 The first two hours 46
12.2 Phases 47
12.3 Who does what 48
12.4 Working rules 48

13 Publication Plan 50
13.1 What is publishable 50
13.2 Where 50
13.3 Outline 51
13.4 Ethics and licences 51

14 Syllabus Map 52

A A DesignSpec 55

References 57

CHAPTER 1

Introduction

1.1 The problem

Most custom clothing in India is still specified the way it was thirty years ago: a customer describes what they want, shows a few photos on their phone, and the tailor writes a handful of numbers on a slip of paper. The description is usually a paragraph in mixed Hindi and English ("kurti chahiye, waist pe fitted, neeche se thoda loose, V neck thoda deep"), and the photos disagree with each other. What reaches the cutting table is the tailor's memory of that conversation. When memory and intent differ, the garment is altered or remade.

This is a large market with almost no software. An analysis of the Periodic Labour Force Survey 2023–24 counts about 12 million custom tailors in India in 2024, roughly one in six manufacturing workers, and 72% of them are women [8]. The platforms that exist (made-to-measure shirt companies, doorstep blouse stitching, home-measurement services) offer fixed option menus and a measurement form. None of them reads free text or a moodboard and turns it into instructions a tailor can cut from.

BESPOKE is that missing layer. It takes two inputs: the customer's words, in English or Hinglish, across one or more messages; and a moodboard, built on an embedded Excalidraw canvas, holding reference images, notes, arrows and circled regions. It produces one output: a *DesignSpec*, a versioned, typed description of the garment by part (neckline, sleeve, bodice, lower body, hem), with values, ease allowances, finished measurements and the source of every value. From the DesignSpec it renders a tailor sheet in English and Hindi. Around it sits a Spark data platform that learns, from completed orders, which defaults and suggestions work.

1.2 What an earlier prototype taught us

Before this project, we built a retrieval prototype for a narrower problem: finding catalogue products for a free-text request. It parsed requests over a 15-slot, 190-value ontology, fused lexical, text and image channels, and ran its corpus side on Spark over 856,667 records. Three of its findings shape BESPOKE.

First, the photo is the strongest signal. On Fashion200K, the FashionSigLIP image channel alone reached MRR 0.219, ahead of every text-only model, so BESPOKE treats images as first-class evidence instead of decoration. Second, explicit structure paid off exactly where language is hard for vector models: exclusions. Appending "but not *c*" made every neural retriever return colour *c*; the structured parser cut those results by more than half. Customers' requests to tailors are full of such constraints ("no embroidery", "not this colour", "deeper than this one"), so BESPOKE keeps structure at its core. Third, a rule parser scored F1 0.981 on 80 hand-written requests, but 80 examples by the same authors say little about real customers. BESPOKE therefore starts with data: real requests, real boards, and practising tailors in the evaluation.

1.3 From retrieval to specification

Retrieval asks "which existing product is closest". A tailor needs something different: a complete, unambiguous instruction for a garment that does not exist yet. That changes four things.

The output becomes part-aware. "Fitted at the waist, loose below" is two values on two parts, not one fit value for the garment. The ontology (Chapter 5) is a graph of garment, parts and typed attributes, some categorical, some ordinal, some numeric in inches.

The input becomes a set. A moodboard holds several images with different roles: one is liked for its sleeve, one is rejected for its colour, and two bound a neckline depth from above and below. Chapter 7 reads the whole board at once.

Uncertainty becomes visible. A spec that silently guesses neckline depth is worse than one that asks, because a neckline cut too deep cannot be raised. The resolver calibrates every value and asks a short question when a construction-critical value is uncertain.

The data platform has a purpose. Every completed order is a labelled example of what was asked, what was made and how it fitted. Chapter 9 turns that stream, together with public retail data, into trends, defaults and suggestions.

1.4 Contributions

The work is planned to deliver five things, each with a measurable exit test in Chapter 12.

1. BESPOKE-SPEC, a benchmark of English and Hinglish garment requests, moodboards and multi-turn revisions, annotated with part-level specifications, released under an open licence.
2. A language engine that converts code-mixed requests into calibrated, part-level specifications and decides when to ask, with every model from TF-IDF to a small LLM evaluated on the same test set.
3. A moodboard interpreter that reads an Excalidraw scene as a graph and pools evidence across images, circled regions, notes and negative references, including interval constraints between references.
4. A tailor sheet and studio, evaluated with practising tailors against the current practice of chat messages and photos.
5. A Spark data platform over public fashion data and the order log, with the MapReduce, DataFrame, DAG and MLlib work the Big Data syllabus asks for.

1.5 How to read this plan

Chapters 2 and 3 review the literature and derive the objectives. Chapter 4 selects models and datasets. Chapters 5 to 8 describe the system, with the language engine and BERT in full detail in Chapter 6. Chapter 9 is the big-data platform and Chapter 10 the engineering workflow, written like developer documentation. Chapter 11 is the evaluation plan, Chapter 12 the phased timeline starting with the first two hours, Chapter 13 the publication plan, and Chapter 14 maps every syllabus topic to the component that covers it.

CHAPTER 2

Literature Review

2.1 Search protocol

Primary sources were restricted to five publishers, following the course guidance: IEEE Xplore (which includes the CVPR, ICCV and WACV proceedings), SpringerLink (including ECCV and LNCS), ScienceDirect, Taylor & Francis and Wiley. The window was 2021 to 2026, with 2025 and 2026 preferred. Every paper below was confirmed against its publisher record or DOI. Queries combined three groups of terms: the task (attribute value extraction, slot filling, sewing pattern generation, garment specification, conversational recommendation), the domain (fashion, apparel, garment, kurta, Indian ethnic wear) and the method (large language model, BERT, CRF, vision-language model, Spark). Work from ACM, ACL and ICLR that no review of this field can skip is discussed separately in Section 2.7.

Fifteen papers form the core review (Table 2.1). For each, we state the technical contribution and the concrete limitation that matters for BESPOKE, in three to four lines, following the format the course asks for.

2.2 Attribute extraction from product text and images

BRINKMANN ET AL. 2024 [2]. *Contribution:* prompts GPT-3.5 and GPT-4 to extract attribute values from product offers and to normalise them (name expansion, generalisation, unit conversion), and introduces the WDC-PAVE benchmark of offers from 59 websites in five categories; GPT-4 reaches F1 91%. *Limitation:* the output is a flat set of attribute–value pairs from seller-written English. Nothing scopes a value to a garment part, and relative customer phrases such as “thoda loose” are outside its normalisation.

Brinkmann et al., ExtractGPT [3]. *Contribution:* compares textual and JSON schema descriptions in zero- and few-shot prompts; GPT-4 reaches 85% average F1, five points above the best fine-tuned encoder baseline, and a fine-tuned GPT-3.5 matches GPT-4 but loses the ability to produce values it never saw. *Limitation:* text only and a flat schema. The unseen-value finding is a warning for the long tail of Indian garment vocabulary, and argues for decoding constrained to an ontology rather than plain fine-tuning.

Zhao et al. 2025 [47]. *Contribution:* builds a multimodal fashion dataset of implicit values (values visible in the image but absent from the text) and a two-step LLM pipeline that first names attributes, then fills values, with visual prompts pointing at key regions. *Limitation:* one image and one text per item. There is no rule for text–image conflict and no record of which source supplied a value, both of which a moodboard needs.

Jia et al., MuJo-SF [21]. *Contribution:* defines multimodal slot filling over a predefined attribute set, releases MAVP (79k description–image pairs), and decides per attribute type whether text or image is the reliable source. *Limitation:* the source decision is fixed per attribute type, not per instance, and slots are always filled. A system that talks to customers needs per-instance routing and the option to leave a slot empty and ask.

2.3 Customer language and fashion text

Cheng et al., ChatPreference [5]. *Contribution:* a domain LLM for understanding customer preferences in apparel concept design, trained in three stages (domain pretraining, supervised fine-tuning on varied preference wordings, and a noise-tolerant DPO variant), demonstrated on men's dress shirts. *Limitation:* one garment category, and the output is a preference reading and a recommendation, not an executable specification with measurements. Code-mixed input is not addressed.

Park et al. 2025 [36]. *Contribution:* a functional–expressive–aesthetic needs taxonomy tagged in reviews by an LLM and fed to a conversational recommender; on trousers, adding need attributes raises success rate and shortens dialogues. *Limitation:* needs are used to rank existing products and never mapped to construction parameters ("leg contouring" never becomes knee ease). Its questions narrow a search; they do not complete a specification.

Shirke et al. 2025 [40]. *Contribution:* an ensemble of HingBERT, HingRoBERTa and HingMBERT for named entity recognition in romanised Hinglish, presented as a cheaper alternative to LLMs. *Limitation:* generic person, place and organisation labels on social-media text; no fashion

entity types, no mixed Devanagari–Roman input, and three encoders triple the inference cost. A single code-mixed encoder fine-tuned on a fashion label set is the natural next step, and it is ours.

2.4 Fashion vision and Indian garments

Chia et al., FashionCLIP [6]. *Contribution:* contrastive image–text fine-tuning of CLIP on a fashion catalogue, giving transferable product representations for retrieval and zero-shot classification, with released weights. *Limitation:* one global embedding per image. It does not say which part carries an attribute, and it was trained on one Western luxury catalogue. We use its successor family as the image encoder and add part-level heads.

Kishore Kumar et al. 2024 [28]. *Contribution:* classifies Indian ethnic wear (92% category accuracy) and, for kurtas, segments parts, detects body keypoints and classifies hem type, sleeve length, hem length and colour (attribute AP50 83%). *Limitation:* the closest prior work to part-level attributes for Indian garments, but for one category and four attributes, with geometric rules that need a garment worn on a posed body. Moodboards contain flat-lays, sketches and crops.

Suman et al., Kouture [41]. *Contribution:* about 20,000 kurta images annotated part-wise for kurta type, sleeve, neckline and hemline, with colour and fabric. *Limitation:* a dataset without a model, labels from listings, and no customer language or measurements. It is our seed for Indian part attributes and a test set for the image branch.

2.5 From design intent to garment patterns

Bian et al., ChatGarment [1]. *Contribution:* fine-tunes a vision-language model to emit a JSON garment description (type, style text and continuous parameters) that a refined GarmentCode program turns into sewing patterns; it handles image, sketch or text input and edits garments over several dialogue turns. *Limitation:* English only, one image per query, an edit to one part leaks into others, no customer measurements, and under-specified requests are filled silently.

Zhou et al., Design2GarmentCode [48]. *Contribution:* a multimodal model synthesises GarmentCode programs from text, images and sketches, checks the simulated result against the

design and corrects itself; 30 professional pattern-makers rated its output. *Limitation:* one standard body, no straps or halter necks, and the input is assumed complete. Nothing asks a question when a dimension is missing.

Nakayama et al., AIpparel [34]. *Contribution:* a multimodal foundation model trained on GCD-MM, over 120,000 garments with text, images, sewing patterns and instruction-edit pairs; a compact pattern tokenisation lets one model generate and edit patterns (stitch accuracy 77.2% against 2.8% for the previous image-to-pattern model). *Limitation:* one image, a fixed set of single-step edit types, a mean body, and a Western garment set.

Gui et al., GenPattern [14]. *Contribution:* SVG-style tokenisation of sewing patterns, a dual-graph module that injects geometric and semantic structure into a multimodal LLM, and a link to a human–robot cutting cell for made-to-order garments; stitch accuracy 86.7%. *Limitation:* a mean panel-vertex error of 2.9 cm is large next to the tolerance at a neckline or armhole, and intent arrives as one text or image with no dialogue.

2.6 Big data for fashion

Goyal et al. 2026 [12]. *Contribution:* a containerised Spark, Hadoop and Hive pipeline that combines ALS collaborative filtering with TF-IDF content filtering on 4.6 to over 40 million Amazon review records, showing sub-linear training-time growth and tolerance to a node failure. *Limitation:* not fashion-specific, content features are bag-of-words text, and rating-based filtering cannot recommend for one-off custom garments. Similarity should be computed over extracted attribute vectors.

2.7 Work beyond the five publishers

Five further works are too close to ignore. TailorCoPilot [42] (UIST 2026) stores sewing patterns as version-controlled states and is the only system that treats revision history as first-class; it serves pattern-makers, not customers. MAI with the FashionMT dataset [4] (ICLR 2025) formalises multi-turn retrieval where a request combines attributes from earlier references, the nearest formal analogue of "sleeve from #2, colour from #1". SIMMC 2.0 [27] (EMNLP 2021) defines dialogue state tracking and reference resolution for fashion shopping. GarmentCode [25] and GarmentCodeData [26] provide the parametric pattern language and 115,000 made-to-measure garments that the pattern-generation papers build on. EasyFashion [37], a September

2026 preprint, combines reference images, text and body photos into designs and patterns with a persistent design state; it is the closest system overall and is not yet peer-reviewed.

2.8 What the literature leaves open

Read together, these papers repeat the same eight limitations.

1. *No part-level binding.* Extractors return one value per attribute per item, so "fitted bodice, flared kalis" collapses to one fit.
2. *One input at a time.* No system combines several references, records which source supplied each value, or resolves text–image conflict.
3. *Closed, Western vocabularies.* Kalidar panels, katori blouses and side slits cannot be expressed in GarmentCode or the extraction schemas.
4. *Under-specification is never detected.* Values are filled silently, without confidence or questions.
5. *Relative terms are not grounded.* "Thoda loose" and "deeper than this one" are never mapped to ease or inches on the customer's body.
6. *Code-mixed NLP stays generic.* Hinglish models use person, place and organisation labels and never feed structured extraction.
7. *Executability is not measured.* Papers report geometric error or user satisfaction, never whether a tailor can cut without calling the customer.
8. *Big-data layers ignore structure.* Spark recommenders use ratings and bag-of-words; nothing aggregates attribute-level demand from orders.

Table 2.1 The core review set, ordered by topic.

	paper	*venue, publisher*	*year*
1	Brinkmann et al., LLMs for extraction and normalisation of attribute values [2]	ADBIS, Springer LNCS	2024
2	Brinkmann et al., ExtractGPT [3]	iiWAS, Springer LNCS	2024
3	Zhao et al., fashion attribute extraction with visual prompting [47]	NLPCC, Springer LNCS	2025
4	Jia et al., MuJo-SF multimodal slot filling [21]	IEEE Trans. Multimedia	2024
5	Cheng et al., ChatPreference [5]	J. Engineering Design, T&F	2026
6	Park et al., FEA needs model for conversational recommendation [36]	Fashion and Textiles, Springer	2025
7	Chia et al., FashionCLIP [6]	Scientific Reports, Springer Nature	2022
8	Kishore Kumar et al., fine-grained Indian ethnic wear [28]	Springer Nature	2024
9	Suman et al., Kouture kurta dataset [41]	LNNS, Springer	2025
10	Bian et al., ChatGarment [1]	CVPR, IEEE	2025
11	Zhou et al., Design2GarmentCode [48]	CVPR, IEEE	2025
12	Nakayama et al., AIpparel [34]	CVPR, IEEE	2025
13	Gui et al., GenPattern [14]	J. Manufacturing Systems, Elsevier	2025
14	Goyal et al., hybrid recommendation on Spark and Hadoop [12]	Discover Computing, Springer	2026
15	Shirke et al., Hinglish NER with HingBERT ensembles [40]	ICCUBEA, IEEE	2025

CHAPTER 3

Research Gap and Objectives

3.1 The five closest systems

Table 3.1 scores the five reviewed systems closest to BESPOKE on the ten capabilities the problem needs. "Part" means the capability is present in a limited form, described in the review above; "n.r." means the paper does not report it.

TABLE 3.1 Capabilities of the five closest systems. The last column is what BESPOKE must deliver.

capability	*ChatGarment*	*Design2GC*	*AIpparel*	*GenPattern*	*MuJo-SF*	*BESPOKE*
free-text input	yes	yes	yes	yes	part	*yes*
code-mixed Hinglish	no	no	no	no	no	*yes*
several reference images at once	no	part	no	no	no	*yes*
region and part references	part	part	no	no	no	*yes*
comparisons between references	no	no	no	no	no	*yes*
multi-turn revision state	part	part	part	no	no	*yes*
clarifying questions	no	no	no	no	no	*yes*
Indian garments	no	no	no	n.r.	n.r.	*yes*
output a tailor can execute	part	part	part	part	no	*yes*
learning from orders and ratings	no	no	no	no	no	*yes*

3.2 The gap

The pattern-generation systems are strong at geometry and weak at intent. They assume one clean English description or one image, fill every parameter silently, and stop at a pattern on a mean body. The extraction systems are strong at turning text into attributes but flatten the garment into one bag of values and never ask. None of them reads a set of references with their relationships, none speaks the language most Indian customers use with their tailor, and none is judged by the people who cut the cloth.

Stated as one sentence: *there is no method that turns a customer's code-mixed language and a multi-reference moodboard into a part-level, calibrated garment specification, asks only the questions that matter for construction, and is validated by practising tailors.*

3.3 Objectives

O1. *To build* a part-aware language engine that converts English and Hinglish garment requests, including multi-turn revisions, into a calibrated DesignSpec, and asks a clarifying question whenever a construction-critical value is uncertain.

O2. *To develop* a moodboard interpreter that reads an entire Excalidraw board (several references, circled regions, notes, rejected references and comparisons between references) into the same DesignSpec, with the source of every value recorded.

O3. *To evaluate* the full system with practising tailors against current chat-and-photo practice, and to build a Spark data platform that learns defaults and suggestions from order outcomes and public fashion data.

3.4 Research questions

RQ1. How far does a code-mixed encoder with a CRF head get on part-level extraction from Hinglish requests, compared with classic taggers, zero-shot extractors and a small LLM with schema-constrained decoding?

RQ2. Does reading the board as a graph of images, regions and notes beat sending the whole board to a vision-language model as one picture? It is the question our retrieval prototype asked, whether explicit structure still helps a strong neural model, asked of a new input.

RQ3. Does calibrated, cost-aware questioning reduce wrong construction-critical values at a fixed question budget?

RQ4. Do tailors working from the sheet need fewer clarifications and make fewer interpretation errors than tailors working from the chat and photos?

3.5 Scope

In scope: women's and men's Indian and Western garments that a single tailor makes from cut cloth (kurta, kurti, anarkali, salwar, churidar, palazzo, saree blouse, lehenga skirt, sherwani, dresses, tops, shirts, skirts, trousers); English, Hinglish and Devanagari Hindi input; tailor sheets in English and Hindi.

Out of scope for the semester: draped garments that are not cut (the saree itself, the dupatta drape), automatic body measurement from photos, automatic sewing-pattern generation for Indian garments (a stretch goal through GarmentCode for Western garments only), and payments or logistics.

CHAPTER 4

Models and Datasets

The rule for choosing is simple: the newest model that is openly licensed, fits the hardware, and has evidence on a task close to ours. The hardware is one laptop RTX 4050 with 6 GB, plus free Kaggle and Colab GPUs. Every version, date and licence below was checked in October 2026.

4.1 Language models

Two choices deserve a reason. The encoder, not the LLM, is the expected main extractor: tagging is a span problem, encoders of 150 to 300M parameters fine-tune comfortably in 6 GB, they run in milliseconds, and our earlier prototype already shipped a 33M encoder that runs in a browser. The LLM is the comparison system and the teacher that produces silver labels. And Qwen3.5-4B is preferred over larger models because it is the newest Apache-licensed multilingual model whose QLoRA training fits the laptop.

4.2 Vision models

Marqo-FashionSigLIP [32] (about 0.2B parameters, Apache-2.0) is the image encoder. It reports text-to-image average recall 0.231 against 0.163 for FashionCLIP 2.0 across seven fashion benchmarks, and it was the strongest single channel in our retrieval prototype. On top of its frozen embeddings we train small attribute heads per slot; zero-shot prompting ("a photo of a kurta with a boat neckline") is the fallback for values without training data. A general vision-language model, Qwen3.5-4B (natively multimodal) or Qwen3-VL-8B on Kaggle, answers open questions about an image and powers the whole-board baseline of RQ2. SAM 2.1 with a text-grounded detector crops garment parts when the customer has not circled one. Image generation for previews (FLUX.2 [klein] 4B, Apache-2.0) is optional and never feeds the specification.

Table 4.1 Language models used, and the role each plays.

model	*size, licence, date*	*role in* BESPOKE
MuRIL [23]	BERT-base, Apache-2.0, 2021	encoder trained on 17 Indian languages and their romanised forms; first candidate for Hinglish tagging
HingRoBERTa, HingMBERT [35]	base, CC BY 4.0, 2022	pretrained on 52.9M romanised Hinglish tweets; second candidate
mmBERT [31]	140M and 307M, MIT, 2025	modern encoder over 1,833 languages, 8K context; third candidate
ModernBERT [44]	149M and 395M, Apache-2.0, 2024	English-only reference point
GLiNER2 [46]	205M to 340M, Apache-2.0, 2025	schema-driven zero-shot extraction baseline; LoRA-adaptable
Qwen3.5-4B [38]	4B, Apache-2.0, March 2026	small LLM for structured generation; 201 languages; QLoRA fits in about 3.5 to 5 GB
Qwen3-Embedding-0.6B	0.6B, Apache-2.0, 2025	maps free phrases (“flowy”) to ontology values
IndicTrans2 [11]	200M to 1B, MIT, 2023	English to Hindi for free-text notes on the tailor sheet

4.3 Datasets

None of these contains what we need most: real customer requests, in Hinglish, labelled by garment part. That is why BESPOKE-SPEC (Chapter 11) is a contribution, not a chore.

Table 4.2 Datasets and what each is used for.

dataset	*size and licence*	*use*
Fashionpedia [20]	48,825 images; 27 categories, 19 parts, 294 attributes; annotations CC BY 4.0	ontology backbone; part-level attribute heads
DeepFashion-MultiModal [22]	44,096 images with parsing, shape, fabric labels and text; non-commercial	text–attribute pairs; image heads
Kouture [41], IndoFashion [39], Atlas [43]	20K kurtas; 106K images in 15 Indian classes; 184K catalogue items	Indian garments, the main gap
Fashion IQ [45]	77,684 images, 60,272 relative captions	comparative language (“longer than”)
SIMMC 2.1 [27]	11,244 dialogues; CC BY-NC-SA	state-tracking pretraining
COMI-LINGUA [7]	125K+ code-mixed Hindi–English instances; CC BY 4.0	Hinglish normalisation and NER
H&M recommendations [16]	1.37M customers, 105,542 articles, about 31.8M transactions	Spark: trends, ALS, text pipelines
Amazon Reviews 2023 [17]	Clothing: 66.0M ratings, 7.2M items	Spark scale; word vectors; review topics
RentTheRunway, ModCloth [33]	192,544 and 82,790 transactions with body data and fit labels	fit model; ease priors
GarmentCodeData [26]	115K garments; CC BY 4.0	stretch goal: spec to sewing pattern

CHAPTER 5

System Design

5.1 Architecture

Figure 5.1 shows the system. Two engines turn the two inputs into evidence. A resolver merges that evidence with the customer's profile into the DesignSpec, decides what to ask, and records where every value came from. The studio, the tailor sheet and the order flow read the spec. Every event lands in a log that the Spark platform turns into trends, priors and suggestions.

5.2 The ontology

The ontology is a typed graph (Figure 5.2). A garment has parts; each part has attributes; each attribute has a type. Categorical attributes take one value from a list (neckline shape). Ordinal attributes take a value from an ordered list and can be compared (neckline depth: shallow < moderate < deep < plunging). Numeric attributes carry a number and a unit, and ordinal fit values map to ease ranges in inches. The types matter because "deeper than #1" only makes sense on an ordered or numeric attribute, and because a tailor needs inches, not adjectives.

The ontology starts from Fashionpedia's attribute groups (silhouette, neckline type, length, opening type, waistline, finishing, textile pattern) and adds Indian garments and parts: kalis, side slits, katori cups, potli buttons, back-neck depth, and embellishments such as zari, gota patti, mirror work and chikankari. Each value carries surface forms in English, romanised Hindi and Devanagari ("dheela, dhila, dheeli, ढीला" all map to *loose*). It is stored as YAML in the repository and versioned like code.

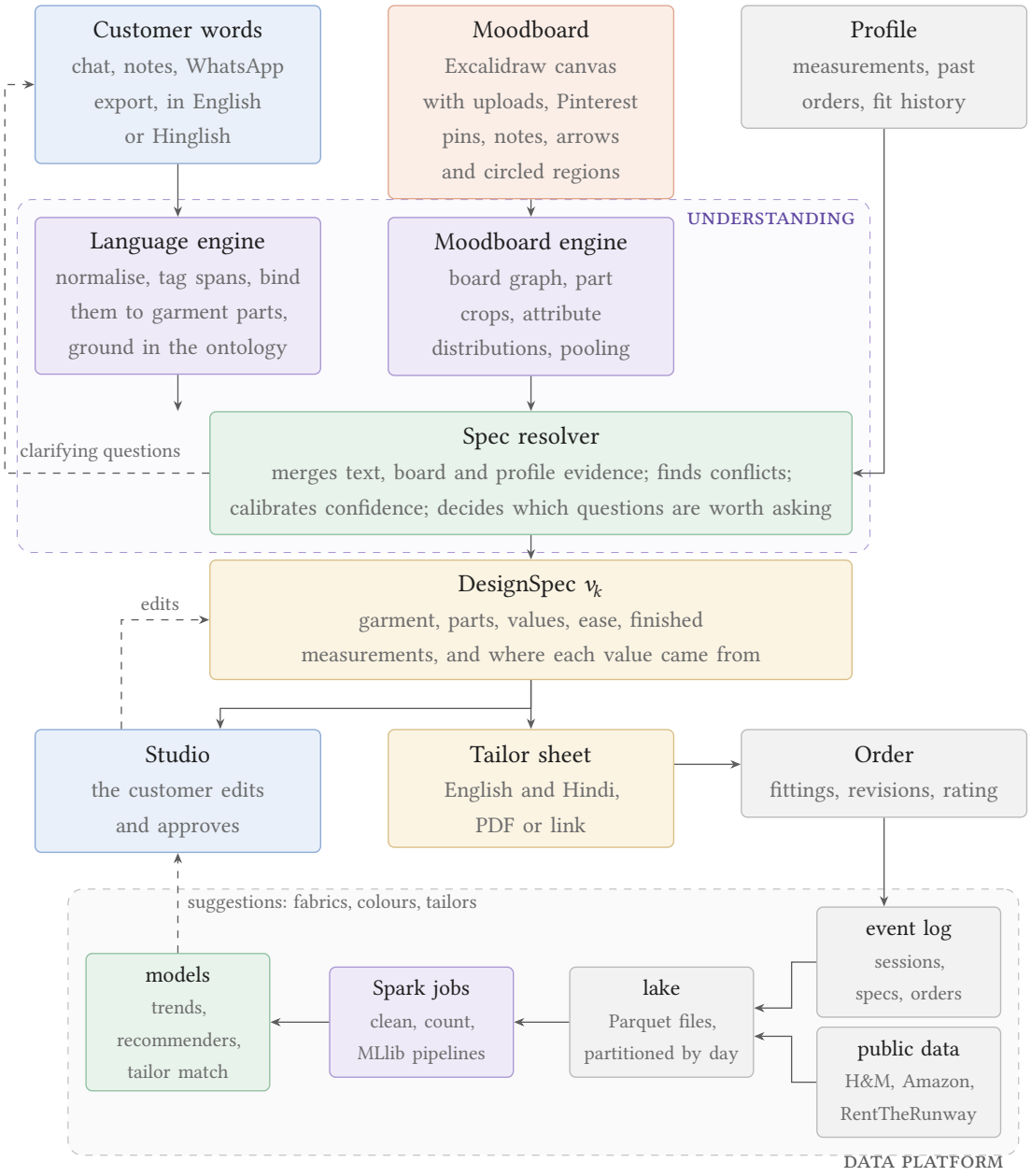


FIGURE 5.1 BESPOKE end to end. Dashed arrows are feedback: questions to the customer, edits from the studio, and suggestions learned by the data platform.

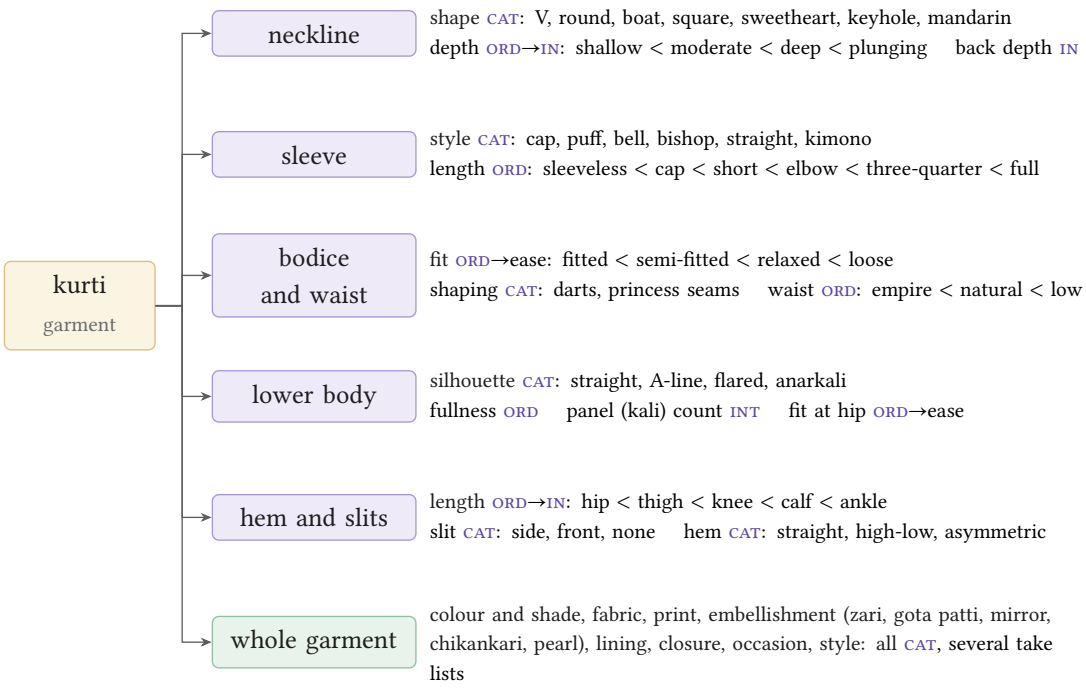


FIGURE 5.2 Part of the ontology for a kurti. Small capitals give the type: CAT categorical, ORD ordinal, IN inches, INT integer; an arrow means the ordinal value is converted to a number.

5.3 The DesignSpec

The DesignSpec is a JSON document validated by a schema generated from the ontology. Every value is an object with five fields: the value itself, a probability, an interval when the value is numeric, a status (*confirmed*, *inferred*, *ask*) and the list of sources. A source is a pointer: a message and character span, an image element id, a circled region, a note, the profile, or a prior from the data platform. Specs are immutable; every change makes a new version, so the order of events can always be replayed. Appendix A gives an example.

Ease is what turns adjectives into cloth. Finished measurement = body measurement + ease, and the fit value chooses the ease. We start from a table with fitted at 1 to 1.5 in, semi-fitted at 2 to 3 in, relaxed at 3 to 4 in and loose at 4 to 6 in, and replace it in Phase 1 with the values our partner tailors actually use. The table is data in the ontology, not code.

5.4 The resolver

For each slot s, the resolver holds evidence from up to five sources: the customer's words, notes on the board, the images, the profile, and a population prior from Spark. It combines them by

weighted log-linear pooling,

$$\log P(s = v) \;=\; \sum_{k} \lambda_k \, \log p_k(s = v) \;-\; \log Z, \tag{5.1}$$

with weights in the order words > notes > images > profile > prior, tuned on development data. When the two strongest sources disagree and both are confident, the slot becomes a conflict and is asked about, not averaged.

What to ask is a decision, not a threshold. Each construction-critical slot has a cost of being wrong, c_s, set with tailors: a neckline cut too deep cannot be raised, while a sleeve cut too long can be shortened. The expected cost of not asking is $c_s \, (1 - \max_v P(s = v))$. The resolver asks at most three questions per round, in order of that expected cost, and for irreversible dimensions proposes the conservative end of the feasible range. Following conformal intent clarification [15], a question offers the values in a calibrated prediction set rather than an open text box: “How deep should the V be: about 6, 6.5 or 7 inches?”

CHAPTER 6

The Language Engine

6.1 The pipeline

The engine runs in five steps: normalise the text, tag spans, bind spans into (part, attribute, value) triples, ground each value in the ontology, and update the conversation state. Each step is built as a ladder of models (Figure 6.1), so every syllabus method is implemented, measured on the same test set, and kept only if it earns its place.

6.2 Normalisation: morphology and variants

Customers write “flowy”, “flowing” and “flows”; and “neeche”, “niche” and “nichey”. Normalisation lowercases, removes character elongation (“loooose”), lemmatises English with a fashion suffix list (*-y*, *-ing*, *-ed*), and maps Hindi inflections that agree with gender and number (“dheela, dheeli, dheele”) to one lemma. Romanised spelling variants are resolved two ways and compared: a variant lexicon mined from HingCorpus with character n-gram similarity, and transliteration to Devanagari with IndicXlit followed by a Hindi lemma lookup. Token-level language identification is kept as a feature for later models.

6.3 Collocations, bag of words and n-grams

Fashion vocabulary is full of multiword units: “boat neck”, “puff sleeve”, “gota patti”, “A-line”. We find them by pointwise mutual information over 7.2 million Amazon clothing titles and the H&M descriptions,

$$\mathrm{PMI}(x, y) = \log \frac{P(x, y)}{P(x)\,P(y)}, \tag{6.1}$$

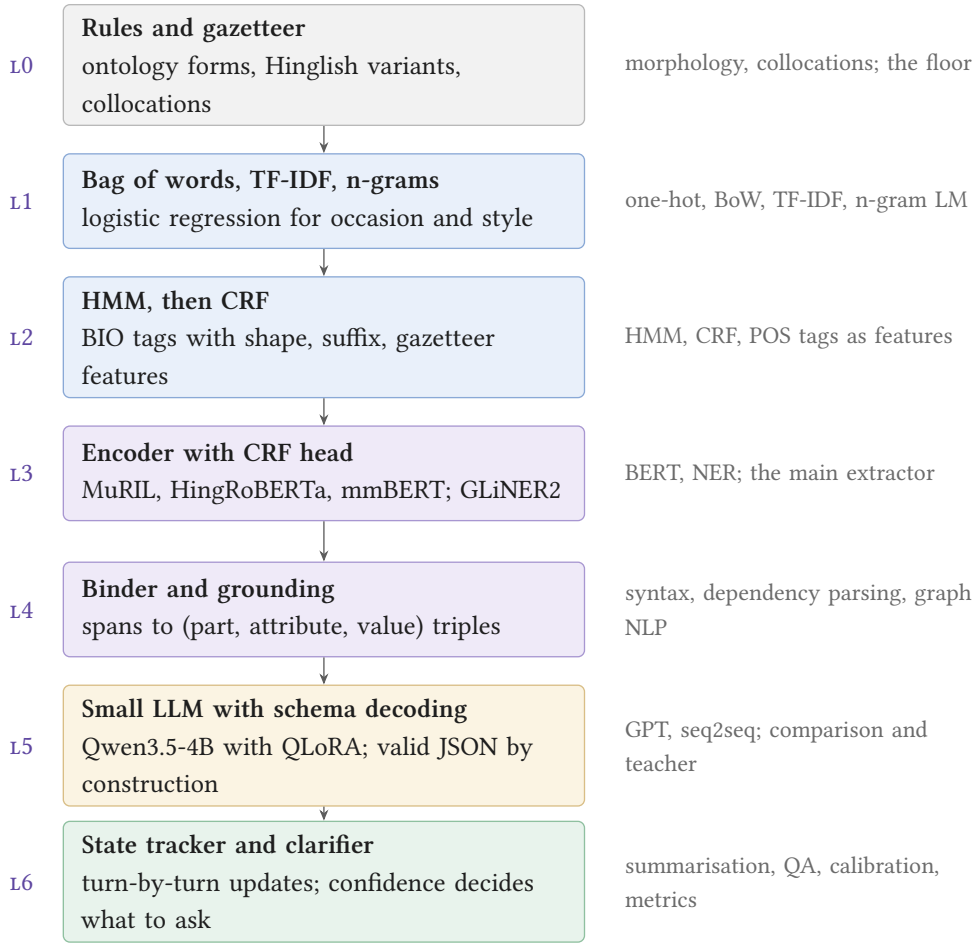


FIGURE 6.1 The model ladder. Left: what each rung is. Right: the syllabus topics it covers and its role.

keeping pairs above a minimum count, and add them to the gazetteer as single tokens.

Whole-request slots (occasion, style, formality) do not need spans. A TF-IDF representation with $\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log(N/\text{df}(t))$ over word unigrams and bigrams, fed to logistic regression, is the baseline, compared against one-hot and raw counts to show what the weighting buys. A Kneser–Ney smoothed trigram language model trained on requests gives two practical tools: autocomplete in the studio ("puff sl…" → "puff sleeves") and a perplexity score that flags requests unlike anything seen, which are routed to the LLM.

6.4 HMM and CRF taggers

Span tagging uses BIO labels over 18 types: garment, part, location ("neeche"), fit, shape, length, dimension word ("deep"), neckline, sleeve, colour, shade, fabric, pattern, embellishment, occa-

sion, intensity (“thoda”, “bahut”), negation (“no”, “mat”, “nahi”) and reference (“#2”, “this one”). Comparison cues (“deeper than”, “se zyada”) are a 19th type.

The first tagger is a hidden Markov model, the generative baseline:

$$P(x, y) = \prod_{i} P(y_i \mid y_{i-1})\, P(x_i \mid y_i), \tag{6.2}$$

trained by counting and decoded with Viterbi. It cannot use overlapping features such as “ends in -ed” or “is in the gazetteer”, which is exactly what the conditional random field [29] fixes. The CRF models $P(y \mid x)$ directly, with features for the word, its lemma, shape, suffixes, gazetteer hits, language tag, POS tag and a window of neighbours. It is the strongest classic baseline and the one the neural models must clearly beat.

6.5 How BERT tags a request

Figure 6.2 follows one request through an encoder with a CRF head.

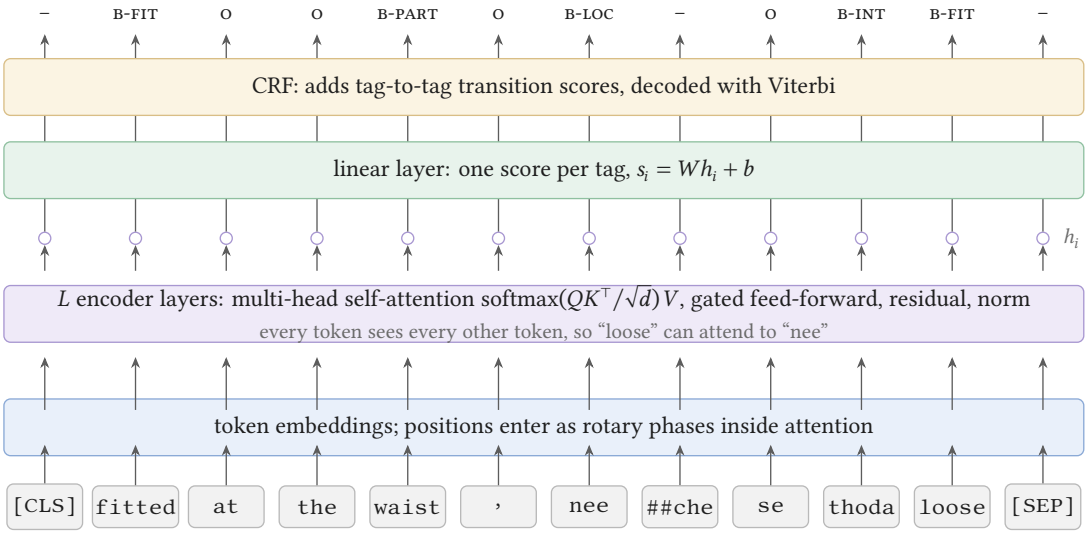


FIGURE 6.2 Token classification with an encoder and a CRF head on “fitted at the waist, neeche se thoda loose”.

TOKENS. The tokenizer splits text into sub-words from a fixed vocabulary, so an unseen romanised word still has a representation (“neeche” becomes “nee” and “##che”). Special tokens mark the start and end.

Embeddings. Each sub-word is looked up in an embedding table to get a vector of width d (768 for base models). Position is added as a learned vector in BERT and MuRIL, or applied as a rotation inside attention in ModernBERT and mmBERT.

Encoder layers. Each of L layers (12 in BERT-base) does two things. Self-attention lets every token build its new vector as a weighted mix of all tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{6.3}$$

where Q, K and V are linear projections of the layer input, computed separately in each of 12 heads. Then a feed-forward network transforms each token on its own. Residual connections and layer normalisation wrap both. Because attention looks both left and right, "loose" can take its meaning from "neeche" before it and the garment after it; this is the advantage over left-to-right models for tagging.

Pretraining. The encoder was pretrained by masked language modelling: hide a share of tokens (15% in BERT [9], 30% in ModernBERT) and predict them from both sides. MuRIL additionally saw translated and transliterated pairs, which is why it is the first candidate for romanised Hindi.

Classification head. For token i with final vector h_i, a linear layer gives one score per tag, $s_i = Wh_i + b$, with $2K + 1$ tags for K span types. Only the first sub-word of each word is labelled; the others get the label -100, which the loss ignores. Plain fine-tuning minimises cross-entropy on these positions.

CRF head. Tags are not independent: I-FIT cannot follow O. A CRF adds a learned transition matrix A and scores a whole tag sequence,

$$\text{score}(x, y) = \sum_i s_i[y_i] + \sum_i A[y_{i-1}, y_i], \qquad P(y \mid x) = \frac{\exp \text{score}(x, y)}{\sum_{y'} \exp \text{score}(x, y')}. \tag{6.4}$$

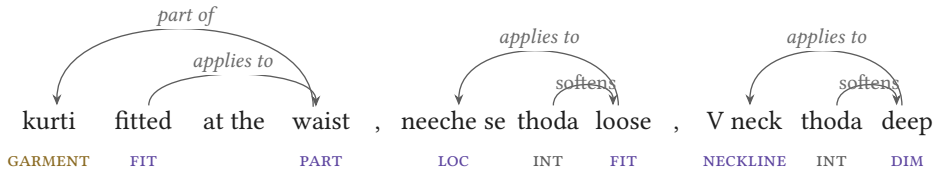
The denominator is computed with the forward algorithm and the best sequence with Viterbi, the same dynamic programmes as the HMM; the difference is that the scores now come from a transformer, not from counts.

Training recipe. AdamW, learning rate 3×10^{-5} for the encoder and 10^{-3} for the head and transitions, 10% linear warm-up, 10 epochs with early stopping on development triple F1, batch 16, maximum length 128, bf16 on the RTX 4050, three seeds. Training data: the human-annotated training split of BESPOKE-SPEC, plus silver data labelled by the LLM teacher and checked on a sample; the test split is never touched by the teacher.

WHICH ENCODER. MuRIL, HingRoBERTa, mmBERT-base and mDeBERTa-v3 are fine-tuned identically and compared on the Hinglish development set. GLiNER2 [46] is run zero-shot with the tag names as its schema, then LoRA-adapted, as the strongest off-the-shelf alternative.

6.6 Binding and grounding

Tags are not yet a specification. In "fitted at the waist, neeche se thoda loose, V neck thoda deep", the engine must decide that *fitted* applies to the waist, *loose* to the part below it, and *thoda* softens *loose* (Figure 6.3).



part	*attribute*	*value*
bodice, waist	fit	fitted
lower body ("neeche" = below the waist)	fit at hip	relaxed, softened by "thoda" (slightly)
neckline	shape, depth	V; deep, softened to moderate-deep

FIGURE 6.3 Binding turns tagged spans into (part, attribute, value) triples.

For English, a dependency parse (Stanza) gives the attachment directly. Parsers trained on standard English or Hindi break on code-mixed text, so the main binder is a pair classifier: for every pair of spans it concatenates their encoder vectors and the distance between them and predicts a relation (*applies-to*, *softens*, *negates*, *compares-with*, none). The resulting graph is checked against the ontology graph: a value must belong to an attribute that its part allows. This is the graph NLP of the syllabus, and it removes impossible triples before they reach the spec.

Grounding maps each value phrase to an ontology value. Exact and variant matches come from the gazetteer. Everything else ("flowy", "ghera wala") is embedded with Qwen3-Embedding and matched to the nearest value description; below a similarity threshold, the phrase is kept as free text for the tailor and flagged.

6.7 Small LLM with schema-constrained decoding

The comparison system treats extraction as sequence-to-sequence generation. Qwen3.5-4B receives the request and emits the DesignSpec JSON directly. A decoder model generates left to right, $P(y) = \prod_t P(y_t \mid y_{<t}, x)$, so on its own it can emit invalid JSON or values outside the ontology. Schema-constrained decoding removes both failure modes: XGrammar [10] or llguidance compiles the JSON schema into a grammar and masks, at every step, every token that would break it. Errors become semantic, never syntactic. Each value must also carry an `evidence` field quoting the request; a post-check rejects values whose evidence is not in the input, which is how we measure and suppress hallucination. The model is fine-tuned with QLoRA on the same training split, and compared zero-shot against GLiNER2.

6.8 Conversation state

Customers revise: "actually make it full sleeves", "colour wahi rakho". Following dialogue state tracking by function calling [30], each turn produces a list of updates (*set*, *remove*, *keep*) to the current spec, and the new spec is the old one with the updates applied. In functional terms this is a fold over the turns, which is how it is written in Scala in Chapter 9. A WhatsApp export is handled the same way: the summary given to the tailor is the final state plus a short changelog, evaluated with state-tracking metrics rather than ROUGE.

6.9 Calibration

A probability is useful only if it is honest. Span confidences from the CRF marginals and value probabilities from the LLM are temperature-scaled on development data, and their expected calibration error is reported [19]. The resolver uses the calibrated values to decide what to ask (Chapter 5).

6.10 Other syllabus tasks in the engine

POS tagging supplies CRF features and is evaluated on its own with an HMM tagger. *Word embeddings*: word2vec trained with Spark MLlib on the product corpus gives neighbours such as "flowy" → "fluid", "loose", "relaxed" for ontology synonyms. *Topic modelling*: LDA in Spark and BERTopic [13] over reviews and requests find themes the ontology does not yet cover. *Sentiment*:

aspect-based sentiment over reviews on fit, fabric and colour feeds size advice, and the same model reads trial feedback ("neck bahut deep ho gaya") as a negative signal on neckline depth. *Machine translation*: ontology terms go to Hindi through a curated glossary, because a tailor's term must be exact; free-text notes go through IndicTrans2. *Question answering*: a tailor can ask the order "sleeve kitni lambi?" and get the value and its source.

CHAPTER 7

The Moodboard Engine

7.1 Why Excalidraw

The customer needs a canvas, not an upload form: a place to drop images, write notes, draw arrows and circle the part they like. Excalidraw is MIT-licensed, embeds as a React component (`@excalidraw/excalidraw` 0.18.1, April 2026), and exposes the whole scene as JSON. tldraw, the obvious alternative, requires a licence key for production use. Embedding in Next.js takes a client component that loads Excalidraw through `dynamic(…, {ssr:false})` and imports its stylesheet.

7.2 Reading the board as a graph

The "Analyse board" button sends the scene: `elements` from `getSceneElements()`, the image bytes from `getFiles()`, and `appState`. Each image element carries a `fileId` (by default the SHA-1 of its bytes, which de-duplicates exact copies for free) and an optional `crop`; each arrow carries `startBinding` and `endBinding` with the ids of the elements it connects; text, frames and groups carry `containerId`, `frameId` and `groupIds`. From these the engine builds a graph (Figure 7.1) with five edge types: *inside* (a circle drawn over an image), *arrow* (a note pointing at an image or circle), *mentions* (a note that names "#2"), *frame* (images grouped as alternatives) and *near* (a note close to an image, used only when nothing else binds it). Near-duplicate images, such as two screenshots of the same pin, are merged by perceptual hash.

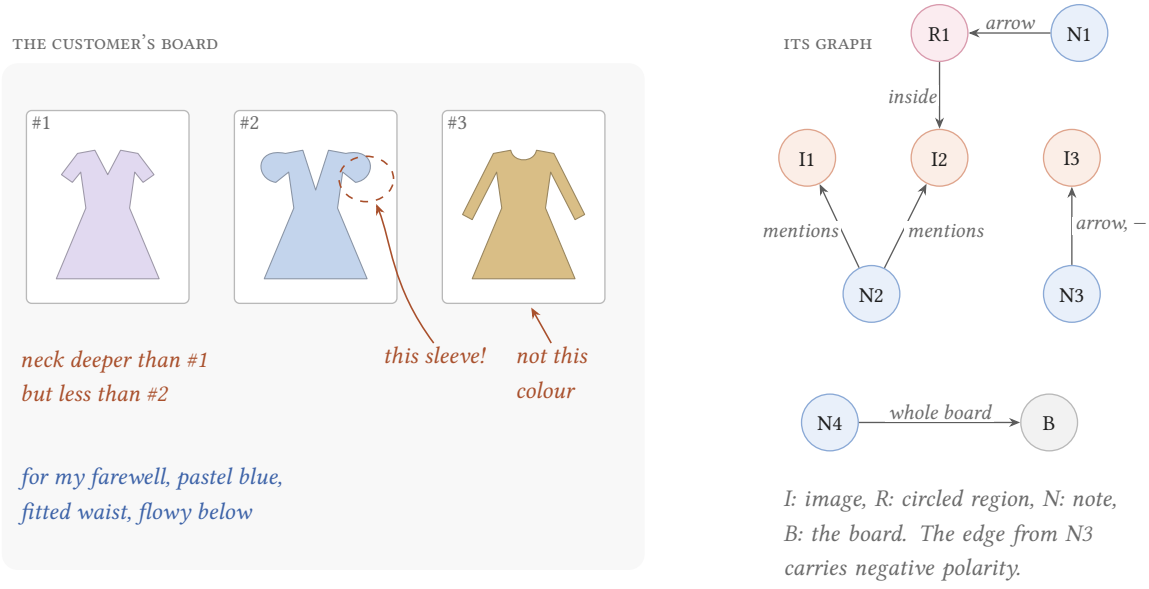


FIGURE 7.1 A customer's board and the graph the engine builds from it.

7.3 Evidence from each element

Figure 7.2 shows the pipeline. *Images* are embedded once with FashionSigLIP; small trained heads give a probability distribution for every slot the garment type allows (neckline shape, sleeve style, silhouette, length and so on). The heads are trained on Fashionpedia, DeepFashion-MultiModal and Kouture, with Indian values labelled by the team where no public label exists. *Circled regions* are cropped and scored by the part-level heads for the part they cover. *Notes* go through the language engine, where "this" resolves to whatever the arrow points at and "#2" to the image labelled 2. A note with negation ("not this colour") gives its target negative polarity for the slots the note names.

7.4 Pooling

For slot s, image i gives a distribution $p_i(v)$, a weight w_i and a polarity $\sigma_i \in \{+1, -1\}$. The board's belief is

$$P_B(s = v) \;\propto\; \exp\Big(\sum_i \sigma_i \, w_i \log p_i(v) \Big). \tag{7.1}$$

The weight encodes salience: an image targeted by a note or circle for this slot gets full weight, an image with no annotation gets half, and an image in a frame of rejected alternatives gets none. A negative reference pushes probability away from the values it shows; if it is confident about

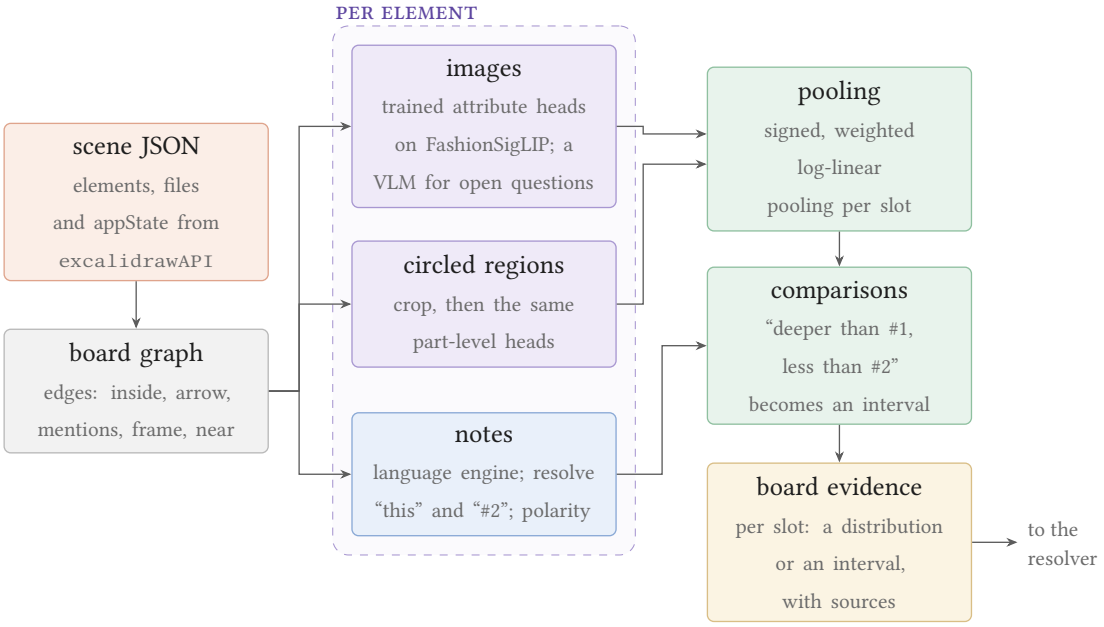


FIGURE 7.2 The moodboard engine. Every value leaves with its sources attached.

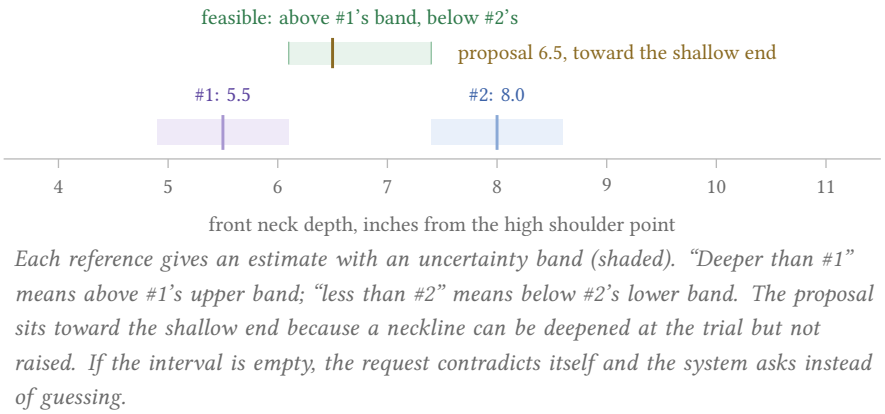
its value, that value becomes a hard exclusion, the mechanism that worked best in our retrieval prototype.

7.5 Comparisons between references

“Neck deeper than #1 but less than #2” is parsed into two constraints on an ordered slot: depth > depth(#1) and depth < depth(#2). Each reference gets an estimate with an uncertainty band. For neckline depth, the estimate is the depth as a fraction of the shoulder-to-bust distance in the image, from body keypoints and the neckline’s lowest point, transferred to the customer’s own shoulder-to-bust measurement. The feasible interval lies between the bands (Figure 7.3). If it is empty, the request contradicts itself and the resolver asks.

7.6 The whole-board baseline

Excalidraw’s own diagram-to-code feature exports a frame as one image plus its texts and sends both to a model. The same approach is our baseline for RQ2: render the board with `exportToBlob`, add every note’s text and the individual images labelled by element id, and ask a vision-language model for the DesignSpec under the same schema constraint. If the structured engine



Each reference gives an estimate with an uncertainty band (shaded). "Deeper than #1" means above #1's upper band; "less than #2" means below #2's lower band. The proposal sits toward the shallow end because a neckline can be deepened at the trial but not raised. If the interval is empty, the request contradicts itself and the system asks instead of guessing.

FIGURE 7.3 A comparison between two references becomes an interval in inches on the customer's body.

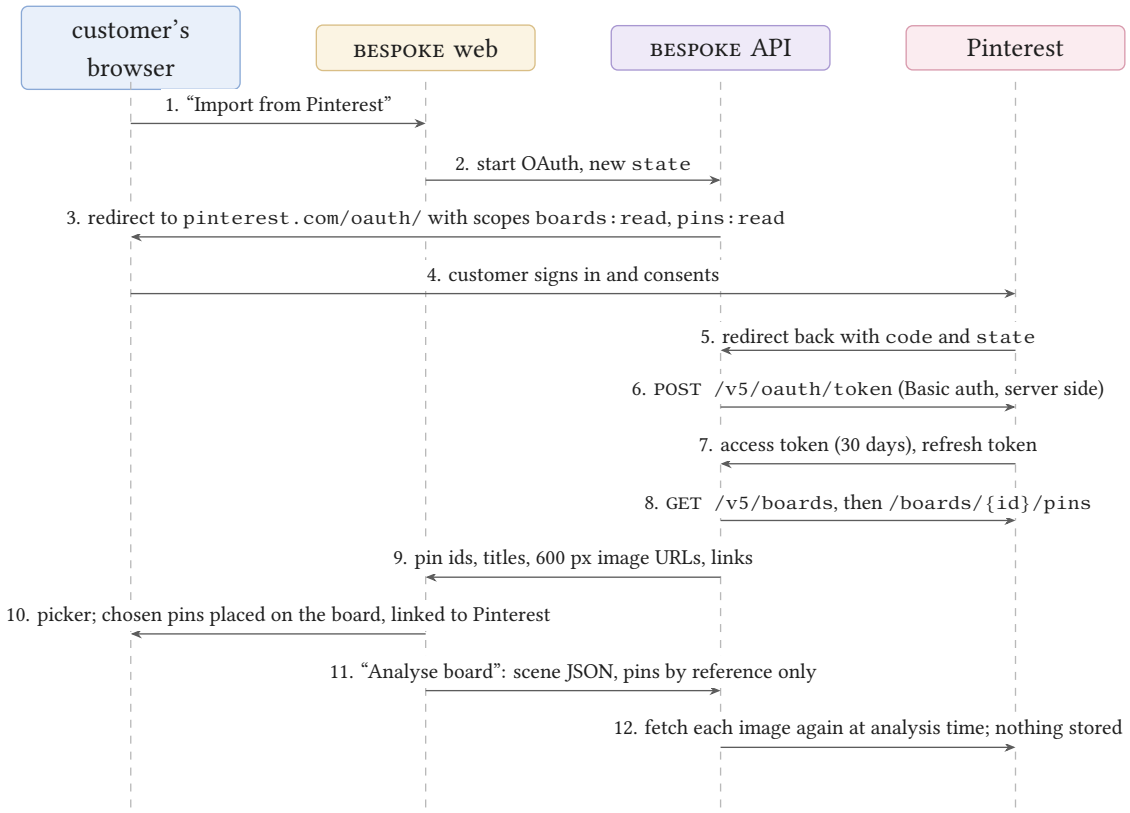
does not beat this on comparison constraints and negative references, the structure is not earning its keep, and we will say so.

7.7 Importing inspiration

UPLOADS. The default path, and the only one used for analysis and training: the customer drops, pastes or shares images they chose. On Android and desktop Chrome and Edge, an installed web app can register as a share target, so "share to BESPOKE" works from the Instagram or Pinterest apps; Safari and Firefox do not support it.

PINTEREST. The Pinterest API v5 supports what a picker needs (Figure 7.4): OAuth with `boards:read` and `pins:read`, `GET /v5/boards`, and `GET /v5/boards/{id}/pins` with bookmark pagination, returning image URLs at 150, 400, 600 and 1200 pixels. Its developer guidelines are strict: content may not be stored (the API is to be called each time), Pinterest data and derivatives must be deleted within 30 days of revocation, pins must link back to Pinterest, and using Pinterest content to train or improve AI models requires Pinterest's express written consent. Apps registered after 14 September 2026 also need Pinterest's approval before reading non-business users' boards. So the integration ships as a picker behind a feature flag, images are fetched at analysis time and never stored, and analysis of pins is switched on only after written approval.

INSTAGRAM. The consumer Basic Display API ended on 4 December 2024. The current APIs serve only a professional account's own media and offer no access to saved posts or other people's posts. Instagram inspiration enters as screenshots or through the share target.



Steps 11 and 12 follow Pinterest's rule to call the API instead of storing its content. Analysing pins at all, and any use for training, needs Pinterest's written consent; until then the import is a picker and the analysed images are the customer's own uploads. Apps registered after 14 September 2026 also need Pinterest's approval before reading non-business users' boards.

FIGURE 7.4 Pinterest import as a picker, following the API's storage and attribution rules.

CHAPTER 8

Studio and Tailor Sheet

8.1 The studio

The studio is where the customer sees the spec as a garment, not as JSON: a parametric flat sketch drawn from the spec (SVG components per neckline, sleeve and silhouette), a short list of open questions, and every value editable by tapping it. Edits are updates to the spec, exactly like a chat message, so they are versioned and logged. An optional preview image can be generated, but the preview is never treated as evidence.

8.2 The tailor sheet

The tailor sheet (Figure 8.1) is the product the tailor sees. It lists each part with one instruction in workshop language, gives finished measurements computed as body measurement plus ease, shows the references with the customer's marks, names the source of every instruction so the tailor knows what is negotiable, and repeats the critical instructions in Hindi. It opens as a link on WhatsApp and prints on one A4 page. Part names and values are translated through a glossary agreed with the partner tailors; only free-text notes use machine translation.

8.3 Sewing patterns, later

For Western garments, the DesignSpec can be compiled to GarmentCode parameters, as ChatGarment and Design2GarmentCode do, giving a sewing pattern and a draped preview. This is a stretch goal. Writing GarmentCode components for kalis, side slits and katori cups is a research project of its own, and the first version of BESPOKE serves tailors who cut from measurements, which is how most Indian custom garments are made.

Order 1024 **Kurti** spec v3 trial 14 Oct delivery 18 Oct

college farewell georgette, cotton lining powder blue, no mustard

PART	INSTRUCTION	FROM
Neckline	V, front depth 6.5 in from high shoulder point	notes, #1, #2
Sleeve	three-quarter, puff at the cap, relaxed	circle on #2
Bodice	fitted, 1.5 in ease at bust and waist	customer words
Lower body	A-line from the waist, 4 in ease at hip	“thoda loose”
Length	42 in; side slits 8 in	profile
Finish	pearl buttons at back neck; no embroidery	notes

MEASUREMENTS, INCHES

bust	waist	hip	shoulder	armhole	sleeve	length
34	28	38	14	16	17	42

REFERENCES



FOR THE TAILOR, IN HINDI

गला: वी आकार, आगे से 6.5 इंच गहरा
कमर पर फिट, नीचे से थोड़ा ढीला (कूल्हे पर 4 इंच ढील)
आस्तीन: तीन-चौथाई, कंधे पर पफ

FIGURE 8.1 A tailor sheet generated from a DesignSpec. Measurements are an example.

CHAPTER 9

The Big Data Platform

9.1 Why this needs big data

A single boutique produces a few hundred orders a year. The platform's value comes from pooling: which ease values led to alterations, which fabrics customers who say "elegant, not flashy" end up choosing, which tailors deliver fitted blouses on time. Until BESPOKE has its own volume, public retail data stands in: the H&M transaction log (about 31.8 million purchases), Amazon clothing reviews (66.0 million ratings), and the RentTheRunway and ModCloth fit records (about 275,000 transactions with body measurements). The same jobs run on the BESPOKE event log as it grows.

9.2 Layout of the lake

Data moves through three layers (Figure 9.1). *Bronze* holds raw files converted to Parquet, partitioned by source and day. *Silver* holds typed, de-duplicated records with text parsed into ontology values; parsing runs inside `mapPartitions` so each executor loads the tagger once. *Gold* holds the tables the product reads: weekly attribute counts, the attribute co-occurrence graph, the user–item matrix and the fit table. Event payloads, which vary by event type, are stored in Spark 4's `VARIANT` column type instead of strings.

Spark 4.2.0 (July 2026) runs on Java 17, 21 or 25 with Scala 2.13 and Python 3.10 or later. Development runs on local Spark on the laptops. Databricks Free Edition is useful for PySpark notebooks, but it runs serverless Spark Connect only, without Scala or the RDD API, so the Scala and MapReduce work stays local.

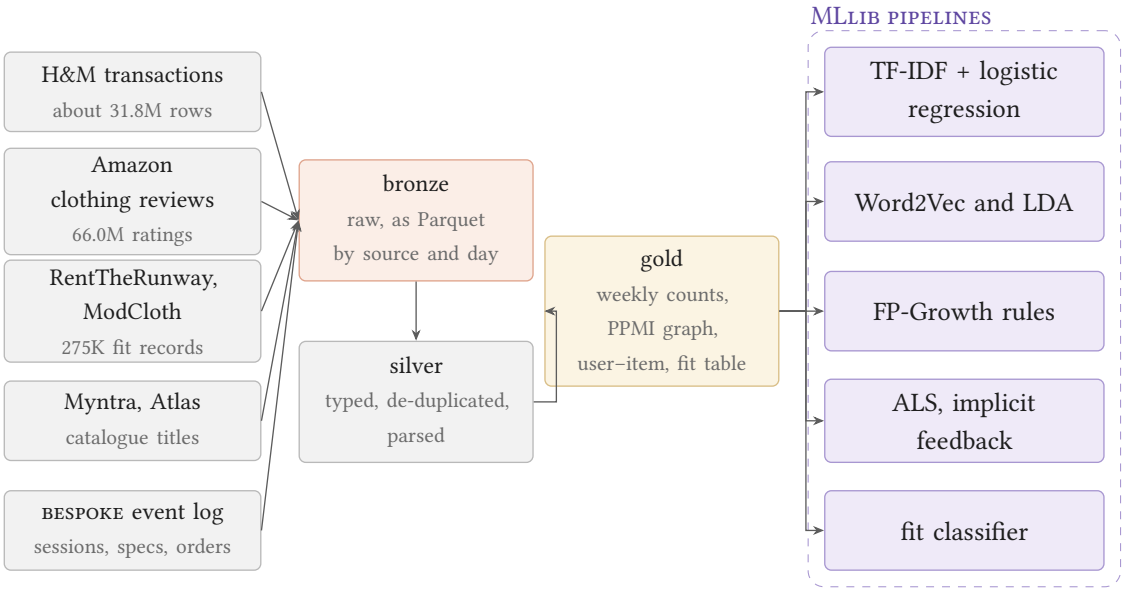


FIGURE 9.1 Sources, lake layers and MLlib jobs.

9.3 Scala: the domain in functional style

The spec's domain model is written in Scala as immutable data, which covers the functional programming unit directly.

```
sealed trait Fit
object Fit {
  case object Fitted     extends Fit
  case object SemiFitted extends Fit
  case object Relaxed    extends Fit
  case object Loose      extends Fit
}

final case class Value(v: String, p: Double, sources: List[String])
final case class Spec(garment: String, slots: Map[String, Value], version:
     Int)

sealed trait Update
final case class SetSlot(slot: String, value: Value) extends Update
final case class Remove(slot: String)                extends Update

def step(s: Spec, u: Update): Spec = u match {
  case SetSlot(k, v) => s.copy(slots = s.slots.updated(k, v), version = s.
    version + 1)
  case Remove(k)     => s.copy(slots = s.slots - k,           version = s.
    version + 1)
}
```

```
// a whole conversation is a fold over its updates
val current: Spec = updates.foldLeft(Spec("kurti", Map.empty, 0))(step)

// ontology ancestors with tail recursion over a cons list
@annotation.tailrec
def ancestors(n: Node, acc: List[Node] = Nil): List[Node] = n.parent
    match {
  case None    => acc
  case Some(p) => ancestors(p, p :: acc)
}
```

`Spec` is immutable, so every version can be kept and replayed; `step` is a pure function, so the fold is the conversation; pattern matching on a sealed trait makes the compiler check that every update type is handled. Higher-order functions and closures appear in the filters (`orders.filter(withinBudget(5000))`); `Futures` fetch image metadata concurrently as the introduction to concurrency.

9.4 MapReduce: attribute co-occurrence

The first job is the textbook one (Figure 9.2). The mapper parses a product text into ontology values and emits each unordered pair with a count of one; the framework shuffles pairs by key; the reducer sums. It is written twice, once with Hadoop Streaming in Python and once as a Scala RDD job with `flatMap` and `reduceByKey`, and the two outputs must match pair for pair. The counts give the positive pointwise mutual information graph between attributes, which the studio uses for suggestions (“customers who choose organza often choose pearl work”).

9.5 DataFrames, lazy evaluation and the DAG

The trends job counts, for every week, how often each ontology value appears in delivered orders (in H&M, in purchased articles) and keeps the top values per slot. Written as DataFrame transformations it does nothing until the final `write`; then Catalyst prunes columns at the scan, pushes the filter into the Parquet reader and splits the plan into stages at each shuffle (Figure 9.3). The report includes the `explain()` output and the Spark UI’s DAG for this job, and a scaling run over 1, 2, 4 and 8 cores and over growing data fractions. A typed `Dataset[Order]` version in Scala shows the compile-time checking that DataFrames lack.

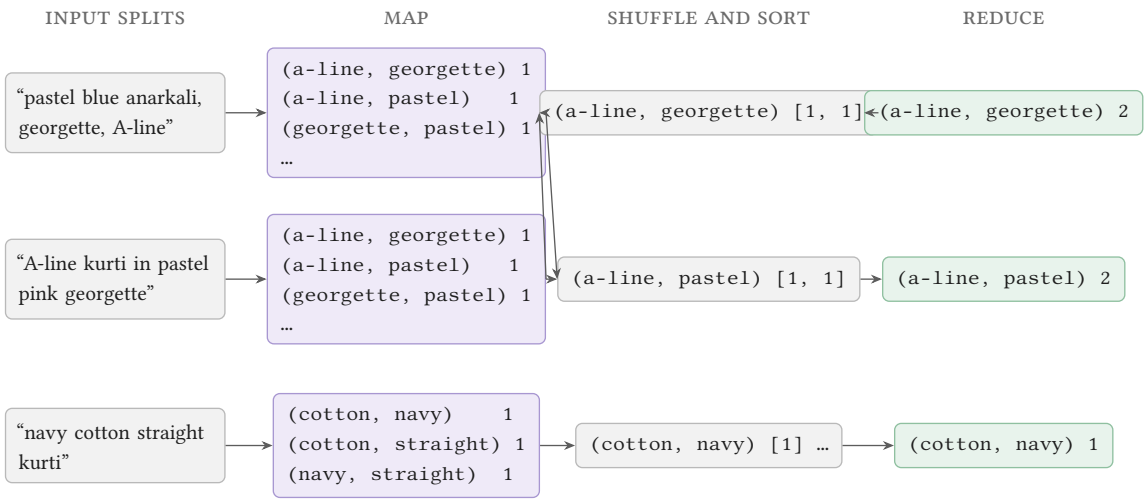


FIGURE 9.2 Attribute co-occurrence as map, shuffle and reduce.

9.6 MLlib pipelines

Five pipelines cover the machine learning unit.

1. *Text classification.* `Tokenizer`, `StopWordsRemover`, `NGram`, `HashingTF`, `IDF` and `LogisticRegression`, predicting H&M garment group from `detail_desc`, tuned with `CrossValidator`. The same pipeline predicts occasion for requests.
2. *Word vectors and topics.* `Word2Vec` over product titles and reviews for ontology synonyms; LDA over clothing reviews for recurring themes.
3. *Association rules.* `FPGrowth` over each article’s or order’s set of ontology values gives rules with support, confidence and lift, such as {pastel, A-line} ⇒ {georgette}.
4. *Collaborative filtering.* Implicit-feedback ALS [18] on H&M purchases, evaluated with MAP@12 on the last week, against a popularity baseline. ALS is the baseline here; the competition’s winners used candidate generation plus a gradient-boosted ranker, and we report that honestly.
5. *Fit.* A classifier on RentTheRunway and ModCloth predicts small, fit or large from body measurements, size and category, and its errors become ease priors per category.

Spark NLP 7.0 runs Hugging Face token classifiers inside Spark pipelines on Spark 4, so the fine-tuned BESPOKE tagger can also parse all 7.2 million Amazon clothing titles at scale.

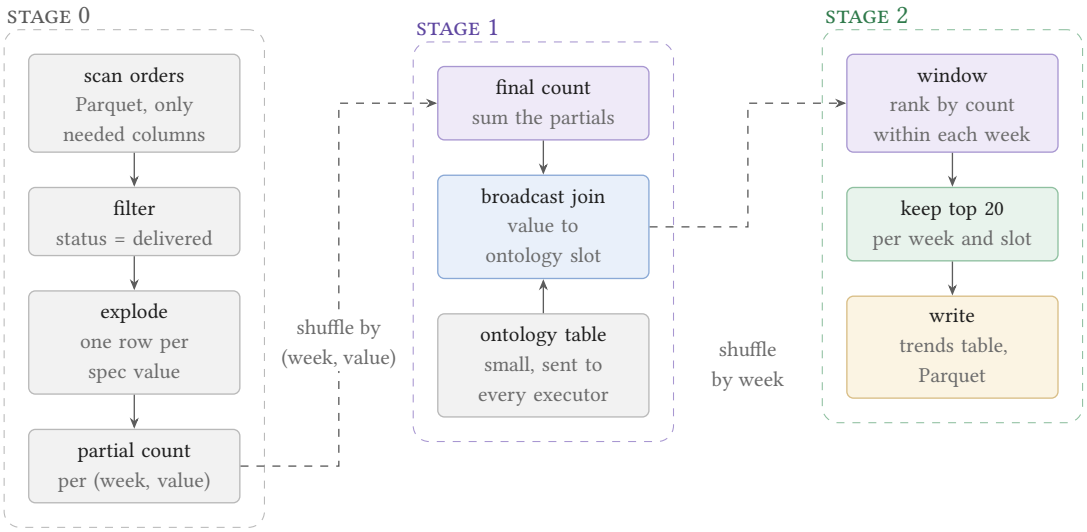


FIGURE 9.3 The weekly trends job as Spark executes it: three stages separated by two shuffles. The small ontology table is broadcast instead of shuffled.

9.7 From data to suggestions

Suggestions combine three signals: content similarity between the current spec and past specs, using FashionSigLIP text embeddings with locality-sensitive hashing in Spark; association rules from FP-Growth; and outcomes, meaning which past choices led to good ratings and no alterations. Tailor matching scores skill coverage for the garment and its parts, capacity, on-time rate and a Bayesian-averaged rating; once enough orders exist it becomes a learned ranker. Figure 9.4 shows the loop that makes this improve with use.

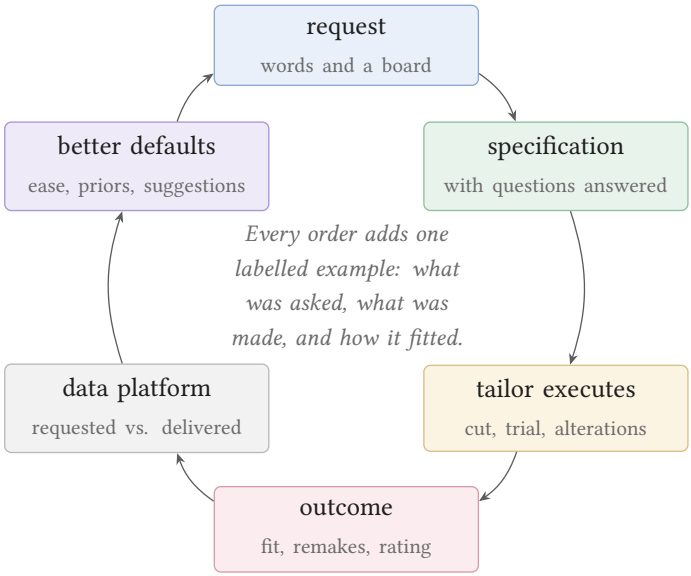


FIGURE 9.4 The data flywheel.

9.8 Privacy

Body measurements and photos are personal data under India's Digital Personal Data Protection Act 2023. The Rules were notified in November 2025, and their main obligations apply from 13 May 2027. The platform is built for that date now: consent tied to a stated purpose, measurements and photos kept encrypted and apart from analytics, pseudonymous ids in the lake, faces blurred before images enter any training set, and deletion that reaches every layer. Delta Lake's DELETE makes the last point practical on Parquet tables.

CHAPTER 10

Engineering Workflow

This chapter is written like developer documentation: what runs where, what each endpoint takes and returns, and how a change moves from a laptop to production.

10.1 Repository

```
onestyle/
  docs/research-plan/   this document (LaTeX, figures, verified sources)
  ontology/             bespoke.yaml, glossary_hi.tsv, ease.yaml
  engine/               Python package: lang/, board/, resolver/, sheet/
  api/                  FastAPI service
  web/                  Next.js app with Excalidraw
  spark/                PySpark jobs, scala/ (sbt), hadoop/ (streaming)
  data/annotations/     BESPOKE-Spec (raw data is downloaded, not
    committed)
  eval/                 benchmark runners, metrics, significance tests
  paper/  report/       manuscripts; every number read from eval/ outputs
```

10.2 Services

The web app (Next.js, deployed on Vercel as a static site) holds the studio and the Excalidraw canvas. The API (FastAPI) owns sessions, specs and orders, and calls the engine in-process. Models run on a GPU machine behind the API: the encoder through ONNX Runtime, FashionSigLIP through `open_clip`, and the LLM through vLLM or llama.cpp with schema-constrained decoding. Postgres stores sessions, specs (JSONB, append-only versions) and orders; object storage holds uploaded images; the event log is written as Parquet for Spark.

10.3 API reference

All endpoints are under `/v1`, take and return JSON, and accept an `Idempotency-Key` header on writes.

endpoint	*does*
`POST /sessions`	start a design session; returns `session_id`
`POST /sessions/{id}/messages`	add customer text; returns the spec delta, new version, and questions
`PUT /sessions/{id}/board`	send the Excalidraw scene and files (`If-Match` on the board version); returns board evidence, new spec version and questions
`POST /sessions/{id}/answers`	answer a question; returns the new version
`GET /specs/{spec_id}?version=`	read a DesignSpec
`GET /specs/{spec_id}/sheet?lang=hi`	tailor sheet as HTML or PDF
`POST /orders`	create an order from a spec, measurements and a tailor
`POST /orders/{id}/events`	cut, trial, alteration, delivered, rated
`GET /recommendations?spec_id=&kind=`	fabrics, colours, designs or tailors
`GET /integrations/pinterest/authorize`	start OAuth; redirects to Pinterest
`GET /integrations/pinterest/callback`	exchange the code server-side; tokens kept encrypted
`GET /integrations/pinterest/boards`, `…/boards/{id}/pins`	picker data, fetched live, never stored

A message call looks like this:

```
POST /v1/sessions/s_81f2/messages
{ "text": "kurti chahiye, waist pe fitted, neeche se thoda loose, V neck
    thoda deep" }

200 OK
{ "spec_version": 2,
  "delta": [
    {"op": "set", "slot": "garment", "value": "kurti", "p": 0.98},
    {"op": "set", "slot": "bodice.fit", "value": "fitted", "p": 0.93},
```

```
    {"op": "set", "slot": "lower.fit_hip", "value": "relaxed", "p": 0.81},

    {"op": "set", "slot": "neckline.shape", "value": "V", "p": 0.96},
    {"op": "set", "slot": "neckline.depth", "value": "moderate-deep", "p":
     0.62,
     "status": "ask"} ],
  "questions": [
    {"id": "q1", "slot": "neckline.front_depth_in",
     "text": "How deep should the V be?", "options": [6, 6.5, 7]} ] }
```

10.4 Events

Every state change emits one event: `event_id`, `ts`, `type`, a pseudonymous `session_id` or `order_id`, `spec_version` and a `payload`. Events are appended, never updated; the lake is rebuilt from them.

10.5 Quality gates

Every pull request runs the unit tests, a parity test that the Python and TypeScript ontology loaders agree (352 fixtures generated from the Python engine), and a regression run on the development split that fails if triple F1 drops by more than one point. Specs are validated against the schema at every boundary. Each evaluation number in the paper is read from a file written by `eval/`, never typed by hand.

CHAPTER 11

Evaluation Plan

11.1 The benchmark: BESPOKE-Spec

BESPOKE-SPEC has three parts, all with part-level gold specifications.

part	*target size*	*source*
requests	1,200	real messages shared with consent by partner boutiques and tailors (target 300); requests written by students for target garments shown as images from Kouture, IndoFashion and DeepFashion-MultiModal (900). At least 40% Hinglish.
boards	150	three to six references each, with notes, circles, a rejected reference and at least one comparison in half of them; gold spec written before the board is analysed
conversations	100	multi-turn threads with revisions, real with consent or role-played from real cases

Two annotators label every test item independently; disagreements are adjudicated, and Cohen's κ is reported per slot, with $\kappa \geq 0.75$ on core slots as the bar for using a slot in evaluation. LLM paraphrases may enlarge the *training* split; the test split contains only human-written text, by people not involved in modelling. Text and specs are released under CC BY 4.0. Board images are referenced by their source dataset ids, not redistributed.

11.2 Metrics

All comparisons report 95% paired bootstrap intervals over test items (2,000 resamples).

what	*metric*
span tagging	strict span F1 (seqeval), per type and micro-averaged
specification	triple F1 over (part, attribute, value); exact match of the whole spec; schema-valid rate
numbers	mean absolute error in inches for depths and lengths
hallucination	share of predicted triples not supported by the input
calibration	expected calibration error; risk–coverage curve and its area
questions	precision (asked and needed) and recall (needed and asked) at a budget of three
conversations	joint goal accuracy and slot F1
boards	slot accuracy; comparison satisfaction rate; violations of rejected references
data platform	MAP@12 for ALS; accuracy and macro-F1 for classifiers; job time against cores and data size

11.3 Baselines and ablations

Every rung of the ladder in Figure 6.1 is a baseline for the rung above it. Beyond that: GLiNER2 zero-shot and LoRA-adapted; Qwen3.5-4B zero-shot and fine-tuned, with and without schema constraints; translate-then-extract (IndicXlit and IndicTrans2 into English, then the English model) against direct code-mixed extraction; the whole-board VLM against the structured board engine; pooling with and without polarity, with and without salience weights; the resolver with a fixed confidence threshold against cost-aware questioning.

11.4 The tailor study

RQ4 is answered with practising tailors, because they are the users the literature never asks. Four to six tailors each receive twenty orders. Each order exists in two forms: the original chat and photos, and the BESPOKE sheet. Each tailor sees each order in one form only, assigned in a counterbalanced design. For each order the tailor writes down what they would cut and lists the questions they would ask the customer. We measure interpretation errors against the gold specification, the number of questions, time to a cutting decision, and a short effort rating, and test the paired differences with the Wilcoxon signed-rank test. A small live pilot of real orders reports alterations and remakes descriptively; it is too small for significance and will be presented that way.

11.5 Targets

These are the exit tests for the phases, set before any result exists.

claim	*target*
language engine, English requests	triple F1 ≥ 0.85
language engine, Hinglish requests	triple F1 ≥ 0.80
calibration	ECE ≤ 0.05 on development data
board engine against the whole-board VLM	higher comparison satisfaction, interval excludes zero
questions	fewer wrong critical values than a fixed threshold at the same budget
tailor study	at least 30% fewer clarifying questions with the sheet

11.6 Threats to validity

Students writing requests for images is not the same as customers describing a dream garment; the real-message subset is reported separately so the gap is visible. Partner tailors who agree to a study may be more digital than average. Fashionpedia and DeepFashion images are mostly Western and studio-lit, which flatters the image heads; Kouture and IndoFashion tests are reported on their own. And a number from one city's tailors does not generalise to India; the paper will say where the tailors work.

CHAPTER 12

Phases and Timeline

12.1 The first two hours

Phase 0 is a working slice, not a plan: by the end of two hours, one sentence and one board produce a spec on screen. Three people work in parallel (Figure 12.1).

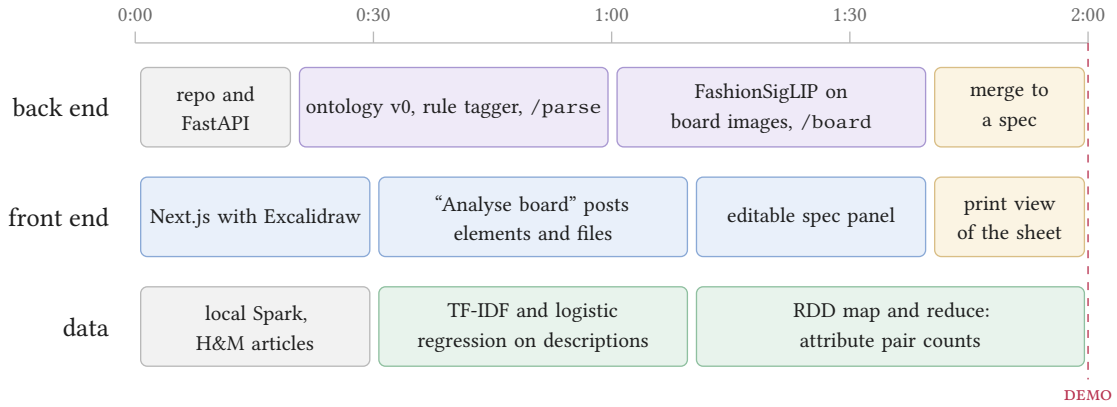


FIGURE 12.1 The first two hours, in ten-minute units.

The back end starts from an empty FastAPI app, writes `ontology/bespoke.yaml` with garment, neckline, sleeve, length, fit and colour, and writes a gazetteer tagger behind a `/parse` endpoint. It then loads FashionSigLIP with `open_clip.create_model_and_transforms('hf-hub:Marqo/marqo-fashionSigLIP')` and scores each board image against prompts for every ontology value. The front end creates the Next.js app, installs `@excalidraw/excalidraw`, wraps it in a client component, and adds an “Analyse board” button that posts `getSceneElements()` and `getFiles()`. The data person installs PySpark with Java 17, loads H&M `articles.csv`, trains a TF-IDF and logistic regression pipeline on `detail_desc`, and writes the attribute-pair counts as an RDD map and reduce. At the two-hour mark the three

pieces meet: the board and a sentence go in, a JSON spec and a printable sheet come out, and the first Spark numbers exist.

12.2 Phases

The semester plan runs for nine weeks from Monday 5 October 2026 (Figure 12.3). Phases overlap on purpose: the data platform runs alongside the engines, and the benchmark grows while the models are built.

Table 12.1 Calendar.

phase	*dates*	*deliverable*
0	4 October	the two-hour slice
1	5–18 October	literature review written; full ontology; 300 annotated requests; interviews with 3–5 tailors on ease, vocabulary and the cost of each error
2	12 October–8 November	the language engine ladder, calibration and state tracking
3	19 October–15 November	board graph, attribute heads, pooling, comparisons, whole-board baseline
4	2–22 November	resolver, studio, tailor sheet in two languages, deployment
5	12 October–22 November	lake, Scala model, MapReduce, DataFrame jobs, MLlib, scaling
6	16 November–6 December	full evaluation, tailor study, paper and course report

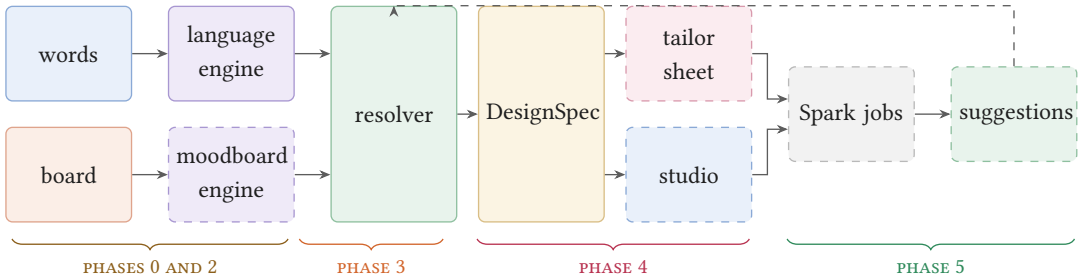


Figure 12.2 How the system grows across the phases. Dashed blocks do not exist yet.

12.3 Who does what

With four people: one owns the language engine and the benchmark's text side; one owns the moodboard engine and the image heads; one owns the data platform and the Scala and Spark work; one owns the product (web app, sheet, tailor study). With three, the product role is shared. Everyone annotates: reading two hundred real requests is the fastest way to understand the problem, and nobody should train a model on data they have not read.

12.4 Working rules

Start every component with the dumbest baseline that runs end to end, and keep it in the results. Overfit a tiny batch before training on everything. Never look at the test split until the final run. Log every experiment's configuration and output file. When a fancy model loses to a simple one, report it; our retrieval prototype reported that its learned ranker and its cross-encoder lost to tuned linear fusion, and that honesty was the most credible part of the paper.

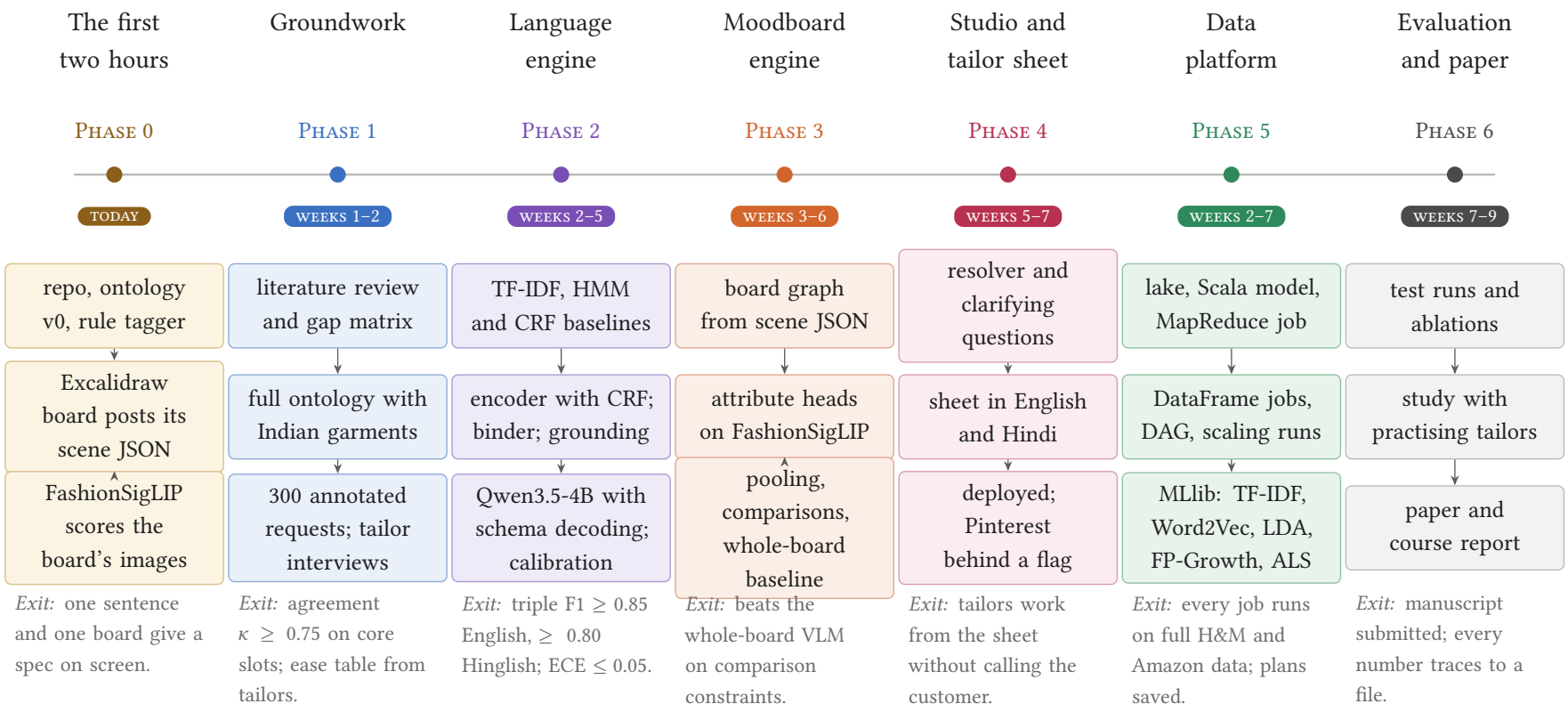


Figure 12.3 All seven phases, with what each delivers and the test it must pass.

CHAPTER 13

Publication Plan

13.1 What is publishable

The publishable unit is the benchmark, the method and the tailor study together. A system description alone is a demo paper; a benchmark with strong baselines and a human study with practising tailors is a journal paper. The paper's claims map to the research questions: the first released part-level benchmark of code-mixed garment requests and moodboards (RQ1, RQ2); evidence on whether explicit board structure beats a strong VLM (RQ2); a cost-aware questioning method with measured effect (RQ3); and the first evaluation of AI-written garment specifications by the people who cut the cloth (RQ4).

13.2 Where

Three journals fit the work's centre of gravity. *Fashion and Textiles* (Springer, open access) has published both of the 2025–2026 conversational and attribute-based fashion papers reviewed here [36, 24]. The *International Journal of Fashion Design, Technology and Education* (Taylor & Francis) suits the tailor study and the design process. *Expert Systems with Applications* or *Engineering Applications of Artificial Intelligence* (Elsevier) suit the method and benchmark. *IEEE Access* is the fast, broad option. The benchmark alone also fits the resources track of an ACL-family venue. Submit to one journal at a time, and choose after the results are in, because the strongest result decides the framing.

13.3 Outline

Introduction with the 12 million tailors and the chat-and-photo practice; related work along the eight limitations of Chapter 2; the ontology and DesignSpec; the language engine; the board engine; the resolver and questioning; the benchmark; experiments for RQ1–RQ3; the tailor study for RQ4; limitations; data and code availability.

13.4 Ethics and licences

Participants consent in writing to the use of their messages, boards and measurements; personal data stays in India-hosted storage and follows the DPDP Act. Faces are blurred in any released image. DeepFashion-MultiModal and SIMMC are non-commercial and are used for research only. Pinterest content is never stored or used for training without Pinterest's written consent. The paper states which tailors were paid and how much.

CHAPTER 14

Syllabus Map

Table 14.1 Text Analytics.

unit	*topic*	*where in* BESPOKE
1	morphology	lemmas, Hindi inflection, spelling variants (Ch. 6)
1	syntax and semantics	binding spans to parts; grounding to ontology values
1	collocations	PMI over 7.2M titles; multiword gazetteer
2	one-hot, bag of words, TF-IDF	occasion and style classifiers, in Python and in MLlib
2	n-gram language model	autocomplete; perplexity for unusual requests
2	neural word embeddings	word2vec in Spark; Qwen3-Embedding for grounding
3	ML and deep learning for NLP	CRF, encoder fine-tuning, LoRA
3	seq2seq, BERT, GPT	encoder with CRF head; Qwen3.5-4B with schema decoding
3	graph NLP	span graph checked against the ontology graph; board graph
3	HMM, CRF	generative and discriminative taggers, Viterbi
3	topic modelling	LDA in Spark; BERTopic on requests and reviews
4	POS tagging, NER	HMM POS tagger; fashion NER as span tagging
4	dependency parsing	Stanza attachment for English requests
4	sentiment analysis	aspect sentiment on reviews and trial feedback
4	machine translation	Hindi tailor sheet: glossary and IndicTrans2
4	question answering	tailor asks the order; questions to the customer
4	summarisation	chat thread to final spec and changelog
4	evaluation and visualisation	seqeval, triple F1, ECE, bootstrap; attention maps, confusion matrices

Table 14.2 Big Data Analytics.

unit	*topic*	*where in* BESPOKE
1	big data characteristics, tools	100M+ records across four public sources and the event log (Ch. 9)
1	Hadoop ecosystem, mapper, reducer	attribute co-occurrence with Hadoop Streaming
2	functional programming in Scala	immutable spec, pure update function, fold over a conversation
2	collections, type hierarchy	sealed traits for ontology types; Map, List, Option
2	higher-order functions, closures	filters and scorers built from functions
2	cons list, tail recursion	ontology ancestors with `@tailrec`
2	object orientation, concurrency	case classes; Futures for concurrent fetches
3	Spark architecture, cluster	driver, executors, local and serverless runs
3	RDDs, transformations, actions	the Scala co-occurrence job
3	DataFrames and Datasets	lake layers; typed `Dataset[Order]`
3	lazy evaluation, optimisation, DAG	weekly trends job, `explain()`, Spark UI, scaling runs
4	MLlib and its algorithms	logistic regression, Word2Vec, LDA, FP-Growth, ALS, fit classifier
4	ML pipeline in Spark	Tokenizer to IDF to classifier with cross-validation
4	case study	fashion retail (H&M) and the custom-tailoring platform

CHAPTER A

A DesignSpec

The spec behind the tailor sheet of Figure 8.1, shortened. Every value carries its probability, status and sources; numbers carry an interval.

```
{
  "spec_id": "sp_1024", "version": 3, "garment": {"value": "kurti", "p":
   0.99,
    "status": "confirmed", "sources": ["msg:1[0:5]"]},
  "parts": {
    "neckline": {
      "shape": {"value": "V", "p": 0.96, "status": "confirmed",
                "sources": ["msg:1[44:50]", "img:e7c2"]},
      "front_depth_in": {"value": 6.5, "interval": [6.1, 7.4], "p": 0.88,
                 "status": "confirmed", "sources": ["note:n2", "img:a91f",
     "img:e7c2", "answer:q1"]}},
    "sleeve": {
      "style": {"value": "puff", "p": 0.84, "status": "inferred", "
    sources": ["region:r1"]},
      "length": {"value": "three-quarter", "p": 0.79, "status": "inferred
    ", "sources": ["img:e7c2"]}},
    "bodice": {"fit": {"value": "fitted", "ease_in": 1.5, "p": 0.93, "
    sources": ["msg:1[14:26]"]}},
    "lower":  {"silhouette": {"value": "A-line", "p": 0.82, "sources": ["
    msg:1", "note:n4"]},
              "fit_hip": {"value": "relaxed", "ease_in": 4.0, "p": 0.81,
     "sources": ["msg:1[28:43]"]}}
  },
  "global": {"colour": {"value": "powder blue", "p": 0.9, "sources": ["
    note:n4"]},
            "occasion": {"value": "farewell", "p": 0.97, "sources": ["
    note:n4"]}},
  "excluded": [{"slot": "colour", "value": "mustard", "sources": ["note:
    n3", "img:5b10"]},
```

```
                {"slot": "embellishment", "value": "embroidery", "sources":
    ["msg:2"]}],
  "measurements_ref": "m_77a0",
  "questions": []
}
```

References

[1] S. Bian et al. ChatGarment: garment estimation, generation and editing via large language models. *CVPR*, 2025, pp. 2924–2934. doi:10.1109/CVPR52734.2025.00278.

[2] A. Brinkmann et al. Using LLMs for the extraction and normalization of product attribute values. *ADBIS*, LNCS 14918, Springer, 2024, pp. 217–230. doi:10.1007/978-3-031-70626-4_15.

[3] A. Brinkmann et al. ExtractGPT: exploring the potential of large language models for product attribute value extraction. *iiWAS*, LNCS 15342, Springer, 2024, pp. 38–52. doi:10.1007/978-3-031-78090-5_4.

[4] Y. Chen et al. MAI: a multi-turn aggregation-iteration model for composed image retrieval. *ICLR*, 2025.

[5] Y. Cheng et al. ChatPreference: a large language model for customer preference understanding in the apparel industry. *Journal of Engineering Design* 37(9), Taylor & Francis, 2026, pp. 3560–3589. doi:10.1080/09544828.2026.2655935.

[6] P. J. Chia et al. Contrastive language and vision learning of general fashion concepts. *Scientific Reports* 12, 18958, 2022. doi:10.1038/s41598-022-23052-9.

[7] LingoIITGN. COMI-LINGUA: an expert-annotated code-mixed Hindi–English dataset. *Findings of EMNLP*, 2025.

[8] Data For India. The rise of custom tailoring (analysis of PLFS 2023–24). dataforindia.com, 2025.

[9] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *NAACL*, 2019.

[10] Y. Dong et al. XGrammar: flexible and efficient structured generation engine for large language models. *MLSys*, 2025.

[11] J. Gala et al. IndicTrans2: towards high-quality and accessible machine translation models for all 22 scheduled Indian languages. *Transactions on Machine Learning Research*, 2023.

[12] S. Goyal et al. A scalable hybrid recommendation framework for e-commerce using distributed big data processing. *Discover Computing* 29, 597, Springer Nature, 2026. doi:10.1007/s10791-026-10544-9.

[13] M. Grootendorst. BERTopic: neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794, 2022.

[14] H. Gui et al. GenPattern: dual-graph enhanced sewing pattern generation via multimodal large language model. *Journal of Manufacturing Systems* 83, Elsevier, 2025, pp. 822–838. doi:10.1016/j.jmsy.2025.11.005.

[15] F. den Hengst et al. Conformal intent classification and clarification for fast and accurate intent recognition. *Findings of NAACL*, 2024.

[16] H&M Group. H&M Personalized Fashion Recommendations. Kaggle competition dataset, 2022.

[17] Y. Hou et al. Bridging language and items for retrieval and recommendation (Amazon Reviews 2023). arXiv:2403.03952, 2024.

[18] Y. Hu, Y. Koren, C. Volinsky. Collaborative filtering for implicit feedback datasets. *ICDM*, 2008.

[19] A. Jagannatha, H. Yu. Calibrating structured output predictors for natural language processing. *ACL*, 2020.

[20] M. Jia et al. Fashionpedia: ontology, segmentation, and an attribute localization dataset. *ECCV*, Springer, 2020.

[21] M. Jia et al. MuJo-SF: multimodal joint slot filling for attribute value prediction of e-commerce commodities. *IEEE Transactions on Multimedia* 26, 2024, pp. 10354–10366. doi:10.1109/TMM.2024.3407667.

[22] Y. Jiang et al. Text2Human: text-driven controllable human image generation. *ACM Transactions on Graphics* 41(4), 2022.

[23] S. Khanuja et al. MuRIL: multilingual representations for Indian languages. arXiv:2103.10730, 2021.

[24] L. Khleborodova, E. T. Kim. From prompting to attribute-based interaction: a taxonomy-driven framework for fashion image generation. *Fashion and Textiles* 13, 27, Springer, 2026. doi:10.1186/s40691-026-00477-8.

[25] M. Korosteleva, O. Sorkine-Hornung. GarmentCode: programming parametric sewing patterns. *ACM Transactions on Graphics* 42(6), 2023. doi:10.1145/3618351.

[26] M. Korosteleva et al. GarmentCodeData: a dataset of 3D made-to-measure garments with sewing patterns. *ECCV*, LNCS, Springer, 2024. doi:10.1007/978-3-031-73027-6_7.

[27] S. Kottur et al. SIMMC 2.0: a task-oriented dialog dataset for immersive multimodal conversations. *EMNLP*, 2021, pp. 4903–4912.

[28] R. Kishore Kumar et al. A robust approach for categorizing and fine-grain classification of Indian ethnic wear. *Algorithms for Intelligent Systems*, Springer Nature, 2024, pp. 37–66. doi:10.1007/978-981-99-8398-8_4.

[29] J. Lafferty, A. McCallum, F. Pereira. Conditional random fields: probabilistic models for segmenting and labeling sequence data. *ICML*, 2001.

[30] Z. Li et al. Large language models as zero-shot dialogue state tracker through function calling. *ACL*, 2024.

[31] M. Marone et al. mmBERT: a modern multilingual encoder with annealed language learning. arXiv:2509.06888, 2025.

[32] Marqo. Marqo-FashionCLIP and Marqo-FashionSigLIP. Model release and benchmark, huggingface.co/Marqo, 2024.

[33] R. Misra, M. Wan, J. McAuley. Decomposing fit semantics for product size recommendation in metric spaces. *RecSys*, 2018.

[34] K. Nakayama et al. AIpparel: a multimodal foundation model for digital garments. *CVPR*, 2025, pp. 8138–8149. doi:10.1109/CVPR52734.2025.00762.

[35] R. Nayak, R. Joshi. L3Cube-HingCorpus and HingBERT: a code mixed Hindi–English dataset and BERT language models. *WILDRE-6 at LREC*, 2022.

[36] H. Park et al. Application of the FEA consumer needs model in e-commerce: a conversational recommendation system for fashion products. *Fashion and Textiles* 12, 24, Springer, 2025. doi:10.1186/s40691-025-00435-w.

[37] H. Qu et al. EasyFashion: a human-AI co-creation system for personalized fashion design and sewing pattern generation. arXiv:2609.18483, 2026 (preprint).

[38] Qwen Team. Qwen3.5 model family. huggingface.co/Qwen, 2026.

[39] P. S. Rajput, S. Aneja. IndoFashion: apparel classification for Indian ethnic clothes. *CVPR Workshops*, 2021. doi:10.1109/CVPRW53098.2021.00440.

[40] M. Shirke et al. Robust named entity recognition in Hinglish code-mixed text using ensemble of pre-trained transformers. *ICCUBEA*, IEEE, 2025. doi:10.1109/ICCUBEA65967.2025.11283863.

[41] S. Suman et al. Kouture: a comprehensive dataset for women's kurtas. *Fifth Congress on Intelligent Systems*, LNNS, Springer Nature, 2025, pp. 421–435. doi:10.1007/978-981-96-2697-7_31.

[42] Y. Sun et al. TailorCoPilot: enabling agentic pattern making with version-controlled state tracking. *UIST*, 2026, to appear. arXiv:2608.25462.

[43] V. Umaashankar et al. Atlas: a dataset and benchmark for e-commerce clothing product categorization. *SIGIR eCom*, 2020.

[44] B. Warner et al. Smarter, better, faster, longer: a modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *ACL*, 2025.

[45] H. Wu et al. Fashion IQ: a new dataset towards retrieving images by natural language feedback. *CVPR*, 2021, pp. 11307–11317. doi:10.1109/CVPR46437.2021.01115.

[46] U. Zaratiana et al. GLiNER2: an efficient multi-task information extraction system with schema-driven interface. *EMNLP System Demonstrations*, 2025.

[47] Z. Zhao et al. Fashion-related attribute value extraction with visual prompting. *NLPCC*, LNCS, Springer Nature, 2025, pp. 107–118. doi:10.1007/978-981-95-3343-5_9.

[48] F. Zhou et al. Design2GarmentCode: turning design concepts to tangible garments through program synthesis. *CVPR*, 2025, pp. 23712–23722. doi:10.1109/CVPR52734.2025.02208.